REDIS

面试宝典 • 100 道高频真题

从数据结构到 Redis 8.10 • 从缓存一致性到生产级系统设计

题目驱动 • 标准回答 • 原理拆解 • 追问链 • 易错点 • 工程落地

版本基线：Redis Open Source 8.10.1（2026-08）

适用岗位：后端研发 / Java / Go / C++ / 基础架构 / 分布式系统 / SRE

> **编辑说明**
>
> 本书借鉴“面试题驱动、层层追问、总结方法”的学习方式，但不复制《剑指 Offer》的原文、题目或受版权保护的版式。内容围绕 Redis 官方文档与当前工程实践重新组织。

2026 Edition • Redis Interview Field Manual

如何使用这本书

这不是一本按命令字母顺序组织的 Redis 手册，而是一套面向“面试判断力”的题库。建议先尝试在 60 秒内口述答案，再阅读原理拆解；第二遍只看追问链和易错点；第三遍用工程落地部分把知识串成系统设计。

难度标签	面试目标
基础必会	能够给出定义、关键机制和一个正确工程例子。
中高级	能够解释底层机制、复杂度与失效边界。
场景压轴	能够从需求/SLO 出发给方案、取舍、容量和故障演练。

A 推荐回答结构

先给结论：30 秒内说清是什么、为什么。

；

再给机制：指出关键数据结构、线程/进程、协议或状态机。

；

再给边界：说出网络分区、崩溃、Big Key、延迟峰值等失效条件。

；

最后落地：给监控指标、命令、容量或补偿方案。

B 能力地图

> **从“背 Redis”到“设计 Redis”**
>
> 最终目标不是记住 100 个孤立答案，而是形成四条主线：数据结构与内存、缓存与一致性、持久化与高可用、性能与系统设计。面试官改变业务条件时，你仍能沿主线重新推导答案。

目录 • 10 章 / 100 题

01 数据结构与数据建模

Q001–Q010 · 从“会用命令”升级到“按访问模式建模”。选择数据结构时同时考虑操作复杂度、内存、Big Key 与 Cluster 约束。

02 底层编码与内存机制

Q011–Q020 · 理解 SDS、listpack、quicklist、哈希表、跳表以及编码切换，建立 Big Key / Hot Key / 碎片治理能力。

03 TTL、缓存模式与淘汰

Q021–Q030 · 把缓存穿透、击穿、雪崩、一致性与 LRU/LFU 串成完整缓存工程知识链。

04 持久化：RDB、AOF 与恢复

Q031–Q040 · 围绕 RPO/RTO 理解 fork、COW、AOF rewrite、Multi-Part AOF 和 Redis 8.10 BACKUP。

05 复制与 Sentinel 高可用

Q041–Q050 · 掌握异步复制的写安全边界、PSYNC/backlog，以及 Sentinel 的主观/客观下线和选举语义。

06 Redis Cluster 与水平扩展

Q051–Q060 · 从 16,384 slots、MOVED/ASK、Hash Tag 到 Resharding 和故障转移，能解释 Cluster 的一致性边界。

07 事务、Lua、Functions 与分布式锁

Q061–Q070 · 理解 Redis 的原子性到底保证什么；把 WATCH、脚本和锁放进真实的并发/租约问题中。

08 Pub/Sub、Streams 与消息处理

Q071–Q080 · 从 fire-and-forget 到 Consumer Group、PEL、XAUTOCLAIM/XNACK，建立至少一次消费与幂等意识。

09 性能诊断、运维与安全

Q081–Q090 · 把 SLOWLOG、LATENCY、CPU、内存、fork、Swap、ACL/TLS 和可观测性组成排障路径。

10 Redis 8.x 与系统设计压轴题

Q091–Q100 · 覆盖 Redis 8 新数据类型与命令，并以秒杀、排行榜、限流和生产架构完成综合设计。

CHAPTER 01

数据结构与数据建模

从“会用命令”升级到“按访问模式建模”。选择数据结构时同时考虑操作复杂度、内存、Big Key 与 Cluster 约束。

> **本章目标**
>
> 完成 Q001–Q010 后，应能在不依赖命令记忆的情况下解释核心机制，并能够说明至少一个“什么时候不该这么做”的边界。

题号	题目	难度
Q001	Redis 为什么这么快?	基础必会
Q002	Redis 和 Memcached 有什么区别?	基础必会
Q003	Redis 有哪些核心数据类型？分别怎么选?	基础必会
Q004	String 最大能做什么？为什么 String 不只是字符串?	基础必会
Q005	Hash 和 String JSON 应该怎么选?	基础必会
Q006	List 可以实现消息队列吗?	基础必会
Q007	Set 最适合哪些场景?	基础必会
Q008	Sorted Set 为什么能做排行榜?	基础必会
Q009	Sorted Set 为什么通常使用 skiplist，而不是平衡树?	中级
Q010	Bitmap 和 HyperLogLog 分别解决什么问题?	基础必会

Q001 | 基础必会 | 高频真题型 | 原理 • 边界 • 工程

Redis 为什么这么快?

01 面试标准回答 • 先说结论

Redis 快不是单因结果。内存访问、短命令路径、事件驱动网络模型、紧凑数据结构、较少锁竞争，以及 Pipeline 降低 RTT 共同决定了低延迟；现代 Redis 还使用后台线程/进程和 I/O threads，所以“Redis 就是单线程”不准确。

02 原理拆解 • 为什么成立

高分回答要把一次请求拆成网络等待、命令执行和回复发送。

简单 GET/SET 时 RTT 可能比命令本身更贵；

核心命令串行化减少锁竞争，但长命令、Lua、大集合运算也因此会直接拖高全实例尾延迟。

03 高频追问链 • 面试官会继续问什么

追问 1 如果 Redis 已经在内存里，为什么仍会出现 500ms 尾延迟?

追问 2 I/O threads 是否意味着所有命令都能并行执行?

答题抓手：先给性能分解，再讲串行模型的收益与代价，最后补 Pipeline、Big Key/慢命令对 p99 的影响。

04 易错点 / Gotchas

不要这样答

不要只答“因为在内存里”；也不要把“核心命令串行化”说成 Redis 整个进程只有一个线程。

05 工程落地 • 如何把答案变成方案

先给性能分解，再讲串行模型的收益与代价，最后补 Pipeline、Big Key/慢命令对 p99 的影响。

一句话拿分

Redis 快不是单因结果。

Q002 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Redis 和 Memcached 有什么区别?

01 面试标准回答 · 先说结论

Redis 更像内存数据结构服务器，Memcached 更接近纯 KV cache。Redis 提供丰富数据类型、RDB/AOF、Replication、Sentinel、Cluster、Lua/Functions、Streams；Redis 8 还把 Search、JSON、Time Series、概率结构等整合进主发行版。

02 原理拆解 · 为什么成立

选型不该简化为“谁更快”。

纯短值 KV cache 两者都能胜任；

如果业务需要排行榜、局部字段更新、可靠消费、持久化、HA 或复杂查询，Redis 的服务端能力价值更大。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，Redis 和 Memcached 有什么区别的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：面试中先说定位，再用一个具体业务说明为什么选 Redis 或更简单的缓存方案。

04 易错点 / Gotchas

不要这样答

不要说“Redis 一定更快”；也不要只背数据类型数量而不谈产品定位和运维边界。

05 工程落地 · 如何把答案变成方案

面试中先说定位，再用一个具体业务说明为什么选 Redis 或更简单的缓存方案。

一句话拿分

Redis 更像内存数据结构服务器，Memcached 更接近纯 KV cache。

Q003 | 基础必会 | 高频真题型 | 原理 • 边界 • 工程

Redis 有哪些核心数据类型？分别怎么选？

01 面试标准回答 • 先说结论

经典类型包括 String、Hash、List、Set、Sorted Set、Stream，以及 Bitmap、Bitfield、HyperLogLog、Geo。Redis 8 还应知道 JSON、Time Series、Bloom/Cuckoo、Count-Min Sketch、Top-K、t-digest、Vector Set；8.8 新增 Array。

02 原理拆解 • 为什么成立

真正考点是“访问模式 -> 数据结构”。

整体值/计数器选 String；

对象字段频繁局部更新选 Hash；

双端队列选 List；

唯一成员和集合运算选 Set；

按 score 排序选 ZSet；

需要历史、ACK、消费组时选 Streams。

03 高频追问链 • 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，Redis 有哪些核心数据类型？分别怎么选的结论会发生什么变化？

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断？

答题抓手：回答时说明主操作、复杂度、数据规模和失效/迁移风险，比单纯枚举类型更专业。

04 易错点 / Gotchas

不要这样答

不要把所有对象都 JSON 序列化进 String；也不要忽略 Big Key、Cluster 多 key 同 slot 等工程约束。

05 工程落地 • 如何把答案变成方案

回答时说明主操作、复杂度、数据规模和失效/迁移风险，比单纯枚举类型更专业。

一句话拿分

经典类型包括 String、Hash、List、Set、Sorted Set、Stream，以及 Bitmap、Bitfield、HyperLogLog、Geo。

Q004 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

String 最大能做什么？为什么 String 不只是字符串？

01 面试标准回答 · 先说结论

Redis String 是 binary-safe value，可保存文本、JSON、Protobuf、压缩字节，也可作为原子计数器、Bitmap 容器、分布式锁 token、幂等 key。`INCR/DECR/SET NX` 等命令让它成为高并发 primitive。

```
INCR page:view:1001
SET lock:order:1001 7f2d... NX PX 30000
```

02 原理拆解 · 为什么成立

客户端用 `GET -> +1 -> SET` 会产生 lost update，而单个 `INCR` 在 Redis 核心命令语义下是原子的。

大二进制对象虽能存，但网络、复制、持久化和删除成本会随 value 体量放大。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，String 最大能做什么？为什么 String 不只是字符串的结论会发生什么变化？

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断？

答题抓手：对象若总是整体读写，String 最简单；若要字段级读写，再考虑 Hash/JSON。

04 易错点 / Gotchas

不要这样答

不要把 binary-safe 理解成“只支持 UTF-8”；也不要因为能存图片就把大文件塞 Redis。

05 工程落地 · 如何把答案变成方案

对象若总是整体读写，String 最简单；若要字段级读写，再考虑 Hash/JSON。

一句话拿分

Redis String 是 binary-safe value，可保存文本、JSON、Protobuf、压缩字节，也可作为原子计数器、Bitmap 容器、分布式锁 token、幂等 key。

Q005 | 基础必会 | 高频真题型 | 原理 • 边界 • 工程

Hash 和 String JSON 应该怎么选?

01 面试标准回答 • 先说结论

核心取舍是服务端字段级操作 vs 应用层整体序列化。Hash 能 `HGET/HSET` 局部字段；String JSON 简单、跨语言、整体缓存方便；Redis 8 的原生 JSON 又提供文档级局部更新和索引能力。

02 原理拆解 • 为什么成立

小 Hash 常使用紧凑编码，可能很省内存，但超多字段会变成 Big Key。

Redis 8.10 Compact Hash 面向大量共享 schema 的 Hash，通过复用字段名进一步降低内存；

是否更省必须用 `MEMORY USAGE` 和真实 workload 验证。

03 高频追问链 • 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，Hash 和 String JSON 应该怎么选的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：先看读写粒度、schema、对象数量，再选择 String/Hash/JSON；大量同 schema 小对象可关注 8.10 Compact Hash。

04 易错点 / Gotchas

不要这样答

不要为“省 key 数”把所有用户塞进一个 Hash；也不要假设 Hash 一定比 JSON String 省内存。

05 工程落地 • 如何把答案变成方案

先看读写粒度、schema、对象数量，再选择 String/Hash/JSON；大量同 schema 小对象可关注 8.10 Compact Hash。

一句话拿分

核心取舍是服务端字段级操作 vs 应用层整体序列化。

Q006 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

List 可以实现消息队列吗?

01 面试标准回答 · 先说结论

可以。`LPUSH/BRPOP` 或 `BLMOVE` 能实现简单阻塞队列，适合短生命周期任务和简单 worker 模型。但一旦需要 ACK、pending、重试、consumer group、历史回放，就应该比较 Streams，而不是继续在 List 上补协议。

02 原理拆解 · 为什么成立

普通 pop 后消息已从队列拿走，消费者此时崩溃会形成可靠性问题。

可以把消息原子移动到 processing list 再确认删除，但这相当于自己实现 ACK/恢复机制。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，List 可以实现消息队列吗的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：选 List/PubSub/Streams 时用四个问题判断：要不要历史、ACK、重试、消费组。

04 易错点 / Gotchas

> **不要这样答**
>
> 不要看到“队列”就固定回答 List；也不要把 List 的消息语义说成 exactly-once。

05 工程落地 · 如何把答案变成方案

选 List/PubSub/Streams 时用四个问题判断：要不要历史、ACK、重试、消费组。

> **一句话拿分**
>
> 可以。

Q007 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Set 最适合哪些场景？

01 面试标准回答 · 先说结论

Set 的核心是成员唯一和集合代数，适合标签、点赞用户、共同好友、黑白名单、去重。`SINTER/SUNION/SDIFF` 可以把部分集合逻辑下沉到服务器端。

02 原理拆解 · 为什么成立

成员查询通常是期望 O(1)，但大集合的交并差成本与输入规模相关，返回结果本身也可能很大；

Cluster 下多 key 集合运算还要满足同 slot 或由客户端拆分。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，Set 最适合哪些场景的结论会发生什么变化？

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断？

答题抓手：集合题主动补“成员规模上限 + 结果集大小 + Cluster slot”三个限制。

04 易错点 / Gotchas

不要这样答

不要对数百万成员 Set 无界地做全量 `SINTER/SUNION`；也不要只看计算复杂度而忽略网络返回体量。

05 工程落地 · 如何把答案变成方案

集合题主动补“成员规模上限 + 结果集大小 + Cluster slot”三个限制。

一句话拿分

Set 的核心是成员唯一和集合代数，适合标签、点赞用户、共同好友、黑白名单、去重。

Q008 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Sorted Set 为什么能做排行榜?

01 面试标准回答 · 先说结论

ZSet 同时维护 member 唯一性和按 score 的有序关系，因此天然支持更新分数、Top-N、个人排名和范围查询。排行榜、延迟任务、优先级队列都是经典用法。

```
ZADD leaderboard 9520 user:1001
ZREVRANGE leaderboard 0 99 WITHSCORES
ZREVRANK leaderboard user:1001
```

02 原理拆解 · 为什么成立

中小规模榜单可用 `ZADD/ZREVRANGE/ZREVRANK`。

若全球十亿用户放一个 ZSet，它会同时成为 Big Key 与 Hot Key;

需要按地区/赛区/时间窗口/哈希分片维护局部榜，再合并局部 Top-K。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，Sorted Set 为什么能做排行榜的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：系统设计里继续追问“十亿用户怎么办”，回答分片榜 + Top-K merge + 历史分段。

04 易错点 / Gotchas

不要这样答

不要只会背 `ZREVRANGE 0 99`；也不要让一个全局榜无限增长。

05 工程落地 · 如何把答案变成方案

系统设计里继续追问“十亿用户怎么办”，回答分片榜 + Top-K merge + 历史分段。

一句话拿分

ZSet 同时维护 member 唯一性和按 score 的有序关系，因此天然支持更新分数、Top-N、个人排名和范围查询。

Q009 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Sorted Set 为什么通常使用 skiplist，而不是平衡树?

01 面试标准回答 · 先说结论

经典 ZSet 大对象通常是 dict + skiplist：dict 负责按 member 快速定位，skiplist 负责按 score 排序和范围遍历；小 ZSet 可能使用 listpack。

02 原理拆解 · 为什么成立

Skiplist 的查找、插入、删除期望 O(logN)，范围扫描实现自然；

随机层级避免了红黑树那类严格平衡旋转逻辑。

Redis 选择它是工程权衡，不是因为平衡树做不到。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，Sorted Set 为什么通常使用 skiplist，而不是平衡树的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：高分点：解释为什么同时需要“member lookup”和“score order”两种索引能力。

04 易错点 / Gotchas

不要这样答

不要说 skiplist 最坏严格 O(logN)；也不要忘记 dict 是 ZSet 组合实现的重要一半。

05 工程落地 · 如何把答案变成方案

高分点：解释为什么同时需要“member lookup”和“score order”两种索引能力。

一句话拿分

经典 ZSet 大对象通常是 dict + skiplist：dict 负责按 member 快速定位，skiplist 负责按 score 排序和范围遍历；小 ZSet 可能使用 listpack。

Q010 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Bitmap 和 HyperLogLog 分别解决什么问题?

01 面试标准回答 · 先说结论

Bitmap 用 bit 表示精确布尔状态，适合签到、活跃标记；HyperLogLog 用概率算法估计基数，适合 UV 等“只要近似唯一数量、不需要成员列表”的场景。

02 原理拆解 · 为什么成立

Bitmap 空间与最大 offset 相关，ID 很稀疏会浪费；

HLL 不能恢复成员，而且有估计误差。

需要精确集合和后续成员运算时应使用 Set 等结构。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，Bitmap 和 HyperLogLog 分别解决什么问题的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：先问业务要成员本身还是只要基数；需要严格结算/计费时避免概率结果。

04 易错点 / Gotchas

不要这样答

不要说 HLL 无误差；也不要忽略 Bitmap 稀疏 ID 的空间问题。

05 工程落地 · 如何把答案变成方案

先问业务要成员本身还是只要基数；需要严格结算/计费时避免概率结果。

一句话拿分

Bitmap 用 bit 表示精确布尔状态，适合签到、活跃标记；HyperLogLog 用概率算法估计基数，适合 UV 等“只要近似唯一数量、不需要成员列表”的场景。

CHAPTER 02

底层编码与内存机制

理解 SDS、listpack、quicklist、哈希表、跳表以及编码切换，建立 Big Key / Hot Key / 碎片治理能力。

> **本章目标**
>
> 完成 Q011–Q020 后，应能在不依赖命令记忆的情况下解释核心机制，并能够说明至少一个“什么时候不该这么做”的边界。

题号	题目	难度
Q011	Redis String 底层为什么不用普通 C 字符串?	中级
Q012	embstr、raw、int 有什么区别?	中级
Q013	List 的底层结构是什么?	中级
Q014	Hash 的底层结构是什么?	中级
Q015	Redis 为什么会自动改变 encoding?	中级
Q016	什么是 Big Key?	基础必会
Q017	Big Key 如何治理?	中级
Q018	什么是 Hot Key?	基础必会
Q019	Hot Key 怎么解决?	中级
Q020	Redis 为什么可能出现内存碎片?	中级

Q011 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Redis String 底层为什么不用普通 C 字符串?

01 面试标准回答 · 先说结论

Redis 使用 SDS 思想而不是裸 C 字符串，核心是二进制安全、O(1) 长度、容量管理和更安全的扩容。SDS 能保存包含 `\0` 的字节序列，也无需反复 `strlen()`。

02 原理拆解 · 为什么成立

SDS 记录 len/capacity，用少量预留空间换更低扩容频率；

结合 RESP 的明确长度字段，天然适合传输任意二进制 payload。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，Redis String 底层为什么不用普通 C 字符串的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：底层结构题始终连接到外部行为：binary-safe、append 性能、内存占用。

04 易错点 / Gotchas

不要这样答

不要只背“防止缓冲区溢出”，也不要把 SDS 等价成 Java String。

05 工程落地 · 如何把答案变成方案

底层结构题始终连接到外部行为：binary-safe、append 性能、内存占用。

一句话拿分

Redis 使用 SDS 思想而不是裸 C 字符串，核心是二进制安全、O(1) 长度、容量管理和更安全的扩容。

Q012 | 中级 | 高频真题型 | 原理 · 边界 · 工程

embstr、raw、int 有什么区别?

01 面试标准回答 · 先说结论

Redis String 会按值选择 encoding：整数可直接用 int；短字符串可用 embstr，让对象头和字符串数据连续分配；更长或修改型字符串使用 raw。核心是小对象减少分配和指针跳转。

02 原理拆解 · 为什么成立

`OBJECT ENCODING key` 可观察当前编码。

embstr 更有 cache locality，但内部阈值属于实现细节，版本可能变化；

业务不能依赖具体 encoding 作为 API 契约。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，embstr、raw、int 有什么区别的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：面试重点应是“为什么要有小对象优化”，而不是只背 44 字节之类实现阈值。

04 易错点 / Gotchas

不要这样答

不要把当前阈值背成永久规范；不要误以为不同 encoding 对外语义不同。

05 工程落地 · 如何把答案变成方案

面试重点应是“为什么要有小对象优化”，而不是只背 44 字节之类实现阈值。

一句话拿分

Redis String 会按值选择 encoding：整数可直接用 int；短字符串可用 embstr，让对象头和字符串数据连续分配；更长或修改型字符串使用 raw。

Q013 | 中级 | 高频真题型 | 原理 · 边界 · 工程

List 的底层结构是什么?

01 面试标准回答 · 先说结论

现代 Redis List 使用 quicklist 组织多个 listpack 节点。这样保留双端链表操作便利，又避免“一元素一个链表节点”的指针和 allocator 元数据开销。

02 原理拆解 · 为什么成立

listpack 连续布局有更好局部性，但中间更新可能搬移数据；

quicklist 把大 List 分块，避免一个巨型连续数组的扩缩问题。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，List 的底层结构是什么的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：从“需要双端 push/pop + 要控制小元素元数据”解释 quicklist + listpack。

04 易错点 / Gotchas

不要这样答

不要继续回答“Redis List 就是双向链表”；也不要忽略 listpack 的版本演进。

05 工程落地 · 如何把答案变成方案

从“需要双端 push/pop + 要控制小元素元数据”解释 quicklist + listpack。

一句话拿分

现代 Redis List 使用 quicklist 组织多个 listpack 节点。

Q014 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Hash 的底层结构是什么?

01 面试标准回答 · 先说结论

小 Hash 常用 listpack 等紧凑编码；字段数量或值大小超过阈值后转为 hashtable。对外抽象不变，内部布局会随规模改变。

02 原理拆解 · 为什么成立

紧凑编码省 metadata、cache locality 好，但查找更接近线性；

大 Hash 用哈希表获得期望 O(1) 字段定位，但付出 bucket、指针和碎片成本。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，Hash 的底层结构是什么的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：理解“为什么小 Hash 和超大 Hash 的性能/内存行为完全不同”。

04 易错点 / Gotchas

不要这样答

不要为省 key 把整张表塞进一个 Hash；不要把 listpack 当哈希表。

05 工程落地 · 如何把答案变成方案

理解“为什么小 Hash 和超大 Hash 的性能/内存行为完全不同”。

一句话拿分

小 Hash 常用 listpack 等紧凑编码；字段数量或值大小超过阈值后转为 hashtable。

Q015 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Redis 为什么会自动改变 encoding?

01 面试标准回答 · 先说结论

这是 size-aware optimization：小对象用完整哈希表/跳表会让元数据占比过高；大对象一直用线性紧凑结构又会使查找修改过慢。Redis 在空间、局部性和算法复杂度之间动态折中。

02 原理拆解 · 为什么成立

紧凑结构不一定 CPU 更差，因为连续内存可能提升 cache locality；

阈值调整会改变 CPU/内存曲线，必须用真实 workload benchmark，而不是越大越“省内存”越好。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，Redis 为什么会自动改变 encoding 的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：把 encoding 理解成 Redis 自动做的“小对象省内存，大对象保效率”。

04 易错点 / Gotchas

不要这样答

不要只用大 O 判断；不要极端调阈值却不测延迟和转换成本。

05 工程落地 · 如何把答案变成方案

把 encoding 理解成 Redis 自动做的“小对象省内存，大对象保效率”。

一句话拿分

这是 size-aware optimization：小对象用完整哈希表/跳表会让元数据占比过高；大对象一直用线性紧凑结构又会使查找修改过慢。

Q016 | 基础必会 | 高频真题型 | 原理 • 边界 • 工程

什么是 Big Key?

01 面试标准回答 • 先说结论

Big Key 指 value 体量或成员数量异常大，不是 key 名很长。它会放大命令执行、网络传输、复制、RDB/AOF、Cluster 迁移和内存释放成本。

02 原理拆解 • 为什么成立

几十 MB String、数百万字段 Hash/Set/ZSet 都可能是 Big Key。

即使某命令计算看似 O(1)，返回整个大 value 的网络成本仍与字节数线性相关；

删除大对象也可能因释放大量内存而阻塞。

03 高频追问链 • 面试官会继续问什么

追问 1 Big Key 和 Hot Key 有什么本质区别?

追问 2 为什么一个 Big Key 在删除、复制和迁移时都危险?

答题抓手：生产应定义单 key 大小/成员数预算，并用 `--bigkeys`、`MEMORY USAGE`、抽样 telemetry 持续监控。

04 易错点 / Gotchas

不要这样答

不要看 key 名长度；不要在线用 `KEYS *` 粗暴排查。

05 工程落地 • 如何把答案变成方案

生产应定义单 key 大小/成员数预算，并用 `--bigkeys`、`MEMORY USAGE`、抽样 telemetry 持续监控。

一句话拿分

Big Key 指 value 体量或成员数量异常大，不是 key 名很长。

Q017 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Big Key 如何治理?

01 面试标准回答 · 先说结论

根治 Big Key 要改变数据模型：按时间、用户、分片键拆分；用 `LTRIM/ZREMRANGEBYRANK/XTRIM` 控制历史；删除时优先异步释放；给单 key 设置硬上限和报警。

02 原理拆解 · 为什么成立

Cluster 迁移的管理单位是 slot，但单个 key 本身不能被 slot 再拆，所以 Big Key 会卡 reshard。

拆太碎又会增加 key metadata，需要在可管理性和 key 数量之间平衡。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，Big Key 如何治理的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：方案要同时回答：如何拆、如何查、如何 TTL、如何迁移、如何监控。

04 易错点 / Gotchas

不要这样答

不要只“定期删大 key”而不限制增长源头；也不要把所有拆分 key 用同一 hash tag 集中到 Hot Slot。

05 工程落地 · 如何把答案变成方案

方案要同时回答：如何拆、如何查、如何 TTL、如何迁移、如何监控。

一句话拿分

根治 Big Key 要改变数据模型：按时间、用户、分片键拆分；用 `LTRIM/ZREMRANGEBYRANK/XTRIM` 控制历史；删除时优先异步释放；给单 key 设置硬上限和报警。

Q018 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

什么是 Hot Key?

01 面试标准回答 · 先说结论

Hot Key 指请求量高度集中到少数 key。即使 Cluster 有很多 shard，一个 key 仍只属于一个 slot 和一个 primary，因此热点不会被自动均摊。

02 原理拆解 · 为什么成立

热点可以是读热点、写热点或大结果集热点；

多个 key 因 hash tag/业务分布集中在同一 slot 还会形成 Hot Slot。

应按 shard、slot、key 看流量，而不是只看集群平均 CPU。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，什么是 Hot Key 的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：面试加分点：指出“单 Hot Key 加机器通常无效”，需要本地缓存、副本化、请求合并或重构 key。

04 易错点 / Gotchas

不要这样答

不要把 Hot Key 和 Big Key 混为一谈；也不要认为扩容节点数自动解决单热点。

05 工程落地 · 如何把答案变成方案

面试加分点：指出“单 Hot Key 加机器通常无效”，需要本地缓存、副本化、请求合并或重构 key。

一句话拿分

Hot Key 指请求量高度集中到少数 key。

Q019 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Hot Key 怎么解决?

01 面试标准回答 · 先说结论

读热点优先把读取前移：应用本地缓存、server-assisted client-side caching、CDN、singleflight/request coalescing。必要时构造多个逻辑副本 key，把读流量分散到多个 slots。

02 原理拆解 · 为什么成立

副本化会引入失效一致性：更新时要广播 invalidation、使用版本号或短 TTL；

写热点更难，常需要分桶计数、批量聚合再异步合并。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，Hot Key 怎么解决的结论会发生什么变化?

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断?

答题抓手：热点治理本质是“复制读、聚合写”，同时明确 stale tolerance。

04 易错点 / Gotchas

不要这样答

不要随机复制 key 却没有失效协议；不要把强一致业务改成长 TTL 本地缓存。

05 工程落地 · 如何把答案变成方案

热点治理本质是“复制读、聚合写”，同时明确 stale tolerance。

一句话拿分

读热点优先把读取前移：应用本地缓存、server-assisted client-side caching、CDN、singleflight/request coalescing。

Q020 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Redis 为什么可能出现内存碎片？

01 面试标准回答 · 先说结论

Redis 频繁创建/删除不同大小对象，allocator 以 size class 管理，已释放块未必立即返还 OS，因此 RSS 可能明显高于 `used_memory`。此外 COW、client buffer、replication buffer 也会抬高实际进程占用。

02 原理拆解 · 为什么成立

碎片不等于内存泄漏；

fragmentation ratio 在小实例上还可能因基数小而夸张，应结合绝对 RSS 差值、趋势、allocator 指标和 workload 判断。

03 高频追问链 · 面试官会继续问什么

追问 1 如果数据规模扩大 100 倍，Redis 为什么可能出现内存碎片的结论会发生什么变化？

追问 2 如何用 OBJECT ENCODING / MEMORY USAGE / --bigkeys 等手段验证你的判断？

答题抓手：容量公式至少包含 dataset + metadata + fragmentation + buffers + COW headroom + safety margin。

04 易错点 / Gotchas

不要这样答

不要把 RSS-used_memory 的所有差值都叫碎片；不要忽略持久化时 COW 峰值。

05 工程落地 · 如何把答案变成方案

容量公式至少包含 dataset + metadata + fragmentation + buffers + COW headroom + safety margin。

一句话拿分

Redis 频繁创建/删除不同大小对象，allocator 以 size class 管理，已释放块未必立即返还 OS，因此 RSS 可能明显高于 `used_memory`。

CHAPTER 03

TTL、缓存模式与淘汰

把缓存穿透、击穿、雪崩、一致性与 LRU/LFU 串成完整缓存工程知识链。

> **本章目标**
>
> 完成 Q021–Q030 后，应能在不依赖命令记忆的情况下解释核心机制，并能够说明至少一个“什么时候不该这么做”的边界。

题号	题目	难度
Q021	Redis Key 是怎么过期的?	基础必会
Q022	TTL 到了，Key 一定瞬间从内存消失吗?	基础必会
Q023	Redis 的过期在主从之间怎么保持一致?	中级
Q024	Redis 有哪些 maxmemory 淘汰策略?	基础必会
Q025	LRU 和 LFU 有什么区别?	基础必会
Q026	什么是缓存穿透?	基础必会
Q027	什么是缓存击穿?	基础必会
Q028	什么是缓存雪崩?	基础必会
Q029	Cache Aside 如何保证数据库和 Redis 一致性?	高级
Q030	Redis Client-Side Caching 是什么?	中级

Q021 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Redis Key 是怎么过期的?

01 面试标准回答 · 先说结论

Redis 不为每个 TTL 建独立定时器，而是结合惰性过期和主动过期：访问 key 时发现过期就删；后台周期性抽样带 TTL 的 key 并清理过期项。

```
SET session:42 <value> EX 1800
TTL session:42
```

02 原理拆解 · 为什么成立

这种设计避免海量 timer 的调度/内存成本。

主动过期循环有时间预算，防止服务器把太多时间都花在清理 TTL；

大量 key 同时到期还会形成 CPU 峰值和 DB 回源压力。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：回答时主动说“逻辑可见性与物理内存释放不是同一时刻”。

04 易错点 / Gotchas

不要这样答

不要把过期理解成毫秒级精确回调；也不要把“逻辑过期”与“RSS 立即下降”混为一谈。

05 工程落地 · 如何把答案变成方案

回答时主动说“逻辑可见性与物理内存释放不是同一时刻”。

一句话拿分

Redis 不为每个 TTL 建独立定时器，而是结合惰性过期和主动过期：访问 key 时发现过期就删；后台周期性抽样带 TTL 的 key 并清理过期项。

Q022 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

TTL 到了，Key 一定瞬间从内存消失吗?

01 面试标准回答 · 先说结论

不一定。TTL 到点后从语义上 key 已失效，但实际删除依赖后续访问或主动过期周期；即使 Redis 已 free，allocator 也可能暂时保留内存而不立即返还 OS。

02 原理拆解 · 为什么成立

因此不能用 RSS 是否同步下降判断过期是否生效。

集中 TTL 还可能产生 expire-cycle 延迟事件，TTL jitter 不仅保护数据库，也能减轻 Redis 自身集中清理压力。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：需要精确定时任务时用更合适的调度/有序集合方案，不依赖 key expiration 回调。

04 易错点 / Gotchas

> **不要这样答**
>
> 不要把 TTL 当精确定时任务；看到 RSS 不降也不要立即判断“过期没执行”。

05 工程落地 · 如何把答案变成方案

需要精确定时任务时用更合适的调度/有序集合方案，不依赖 key expiration 回调。

> **一句话拿分**
>
> 不一定。

Q023 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Redis 的过期在主从之间怎么保持一致?

01 面试标准回答 · 先说结论

在复制模式下，primary 负责最终过期语义，并把相应删除操作传播给 AOF 和 replicas。这样避免 primary/replica 因时钟和采样不同各自独立删 key 而产生明显分歧。

02 原理拆解 · 为什么成立

replica 保存 TTL 元数据，但跟随 primary 时不能理解成完全独立决定最终删除；

被提升为 primary 后，它才重新独立处理过期。

异步复制还意味着 replica 可能短时间读到旧值。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：把这题与“Redis replication 默认异步”一起回答，说明一致性边界。

04 易错点 / Gotchas

不要这样答

不要把复制说成同步锁步；不要忽略 replica read 的 stale window。

05 工程落地 · 如何把答案变成方案

把这题与“Redis replication 默认异步”一起回答，说明一致性边界。

一句话拿分

在复制模式下，primary 负责最终过期语义，并把相应删除操作传播给 AOF 和 replicas。

Q024 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Redis 有哪些 maxmemory 淘汰策略?

01 面试标准回答 · 先说结论

常见有 allkeys-lru/lfu/random、volatile-lru/lfu/random/ttl、noeviction。`allkeys-*` 从所有 key 选牺牲者，`volatile-*` 只从设置 TTL 的 key 中选。

02 原理拆解 · 为什么成立

通用 cache 常从 allkeys-lru 或 allkeys-lfu 开始评估；

noeviction 适合不能默默丢业务状态、宁愿显式写失败的场景。

策略选择依访问分布，不存在固定万能答案。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：答题先说明候选集合，再说明 victim 选择算法，最后结合 workload 解释 LRU/LFU。

04 易错点 / Gotchas

不要这样答

不要把 maxmemory 当物理 RAM 上限；不要以为 volatile 策略会自动给无 TTL key 加 TTL。

05 工程落地 · 如何把答案变成方案

答题先说明候选集合，再说明 victim 选择算法，最后结合 workload 解释 LRU/LFU。

一句话拿分

常见有 allkeys-lru/lfu/random、volatile-lru/lfu/random/ttl、noeviction。

Q025 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

LRU 和 LFU 有什么区别?

01 面试标准回答 · 先说结论

LRU 关注最近是否访问，LFU 关注访问频率。近期热点快速变化时 LRU 往往更自然；长期稳定热点时 LFU 可能更适合。Redis 使用近似算法，而不是精确维护全局 LRU 链表。

02 原理拆解 · 为什么成立

一次性扫描型流量可能污染 LRU；

短时爆火后长期沉寂的 key 可能在 LFU 中保留更久。

真实选择应看热点生命周期、频率分布和 hit rate benchmark。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：用“突发热点 vs 稳定热点”举例能快速展示理解。

04 易错点 / Gotchas

不要这样答

不要只背“最近 vs 频率”后结束；不要说 Redis 维护精确全局 LRU linked list。

05 工程落地 · 如何把答案变成方案

用“突发热点 vs 稳定热点”举例能快速展示理解。

一句话拿分

LRU 关注最近是否访问，LFU 关注访问频率。

Q026 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

什么是缓存穿透?

01 面试标准回答 · 先说结论

缓存穿透是大量请求访问本来就不存在的数据，导致每次 cache miss 后继续打 DB。根因通常是无效/恶意 key，而不是热点 key 过期。

02 原理拆解 · 为什么成立

治理可组合参数校验、negative cache、Bloom Filter 和 rate limit。

负缓存 TTL 要短，避免数据后来创建后长期读不到；

Bloom Filter 有 false positive，但能高效挡住“肯定不存在”的请求。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：记忆：不存在的数据=穿透；单热点过期=击穿；大量 key 同时失效=雪崩。

04 易错点 / Gotchas

不要这样答

不要把穿透和击穿混淆；不要把负缓存 TTL 设得过长。

05 工程落地 · 如何把答案变成方案

记忆：不存在的数据=穿透；单热点过期=击穿；大量 key 同时失效=雪崩。

一句话拿分

缓存穿透是大量请求访问本来就不存在的数据，导致每次 cache miss 后继续打 DB。

Q027 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

什么是缓存击穿?

01 面试标准回答 · 先说结论

缓存击穿是一个高热点 key 失效后，大量并发请求同时 miss 并回源，数据库被同一份数据的重复加载打爆。核心问题是并发回源没有合并。

02 原理拆解 · 为什么成立

可用 singleflight/mutex、stale-while-revalidate、逻辑过期、refresh-ahead。

锁本身会带来排队和故障问题，热点上不应无脑把所有请求都阻塞在一个分布式锁上。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手: 高分答案：singleflight 的价值是“合并回源”，不是让 DB 本身变快。

04 易错点 / Gotchas

不要这样答

不要把所有 cache miss 都加全局锁；锁若无 TTL/owner 校验又会制造新故障。

05 工程落地 · 如何把答案变成方案

高分答案：singleflight 的价值是“合并回源”，不是让 DB 本身变快。

一句话拿分

缓存击穿是一个高热点 key 失效后，大量并发请求同时 miss 并回源，数据库被同一份数据的重复加载打爆。

Q028 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

什么是缓存雪崩?

01 面试标准回答 · 先说结论

缓存雪崩是大量 key 在短时间同时失效，或 Redis 整体不可用，导致请求集体涌向 DB/下游。它是大范围失效，不是单个热点。

02 原理拆解 · 为什么成立

自然过期可用 TTL jitter、分批预热/刷新、多级缓存；

基础设施故障则还需要限流、熔断、降级、DB 保护和 HA。

只加随机 TTL 解决不了 Redis 整层故障。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手: 成熟回答同时覆盖“批量 TTL 同步到期”和“Redis 层整体故障”两类原因。

04 易错点 / Gotchas

不要这样答

不要只回答“TTL 加随机数”；不要把雪崩、穿透、击穿混在一起。

05 工程落地 · 如何把答案变成方案

成熟回答同时覆盖“批量 TTL 同步到期”和“Redis 层整体故障”两类原因。

一句话拿分

缓存雪崩是大量 key 在短时间同时失效，或 Redis 整体不可用，导致请求集体涌向 DB/下游。

Q029 | 高级 | 高频真题型 | 原理 · 边界 · 工程

Cache Aside 如何保证数据库和 Redis 一致性?

01 面试标准回答 · 先说结论

常见写路径是先更新 DB，再删除 cache。它不是强一致协议，而是最终一致策略；相比“更新 DB 同时更新 cache”，删除更能减少并发旧写覆盖，但 DB commit 后、cache delete 前崩溃仍有脏缓存窗口。

```
READ:  cache -> miss -> DB -> populate cache
WRITE: update DB -> delete cache
```

02 原理拆解 · 为什么成立

读 miss 后回填 cache 也可能与并发更新形成竞态。

增强方案包括删除重试、消息队列、CDC/binlog 驱动失效、版本号/逻辑时间等；

关键是明确允许多长 stale window。

03 高频追问链 · 面试官会继续问什么

追问 1 为什么通常是“更新 DB，再删除缓存”？

追问 2 这个顺序仍有哪些竞态窗口，如何降低?

答题抓手：高分回答最好画两个线程时序，指出至少一个 race window，并说明补偿/CDC。

04 易错点 / Gotchas

不要这样答

不要说“先 DB 后删缓存就完全一致”；不要把 delayed double delete 当万能公式。

05 工程落地 · 如何把答案变成方案

高分回答最好画两个线程时序，指出至少一个 race window，并说明补偿/CDC。

一句话拿分

常见写路径是先更新 DB，再删除 cache。

Q030 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Redis Client-Side Caching 是什么?

01 面试标准回答 · 先说结论

Client-Side Caching 把热点读保存在应用进程本地，并由 Redis server-assisted tracking/invalidation 通知客户端相关 key 发生变化，从而降低 Redis QPS、网络 RTT 和序列化成本。

02 原理拆解 · 为什么成立

它最适合读多写少、低延迟敏感场景。

难点从“取值”转移到本地容量、失效通知、重连后的状态恢复；

本地副本仍不能被当作强一致数据库副本。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：当面试官问“Redis 已经是缓存，为什么还要再缓存一层”，这就是核心答案。

04 易错点 / Gotchas

不要这样答

不要只看命中率而不处理重连/失效；不要把本地缓存的 stale 风险忽略。

05 工程落地 · 如何把答案变成方案

当面试官问“Redis 已经是缓存，为什么还要再缓存一层”，这就是核心答案。

一句话拿分

Client-Side Caching 把热点读保存在应用进程本地，并由 Redis server-assisted tracking/invalidation 通知客户端相关 key 发生变化，从而降低 Redis QPS、网络 RTT 和序列化成本。

CHAPTER 04

持久化：RDB、AOF 与恢复

围绕 RPO/RTO 理解 fork、COW、AOF rewrite、Multi-Part AOF 和 Redis 8.10 BACKUP。

> **本章目标**
>
> 完成 Q031–Q040 后，应能在不依赖命令记忆的情况下解释核心机制，并能够说明至少一个“什么时候不该这么做”的边界。

题号	题目	难度
Q031	RDB 是什么?	基础必会
Q032	SAVE 和 BGSAVE 有什么区别?	基础必会
Q033	什么是 Copy-On-Write?	中级
Q034	AOF 是什么?	基础必会
Q035	AOF 为什么需要 rewrite?	中级
Q036	Redis 7 以后的 Multi-Part AOF 是什么?	中级
Q037	RDB 和 AOF 应该选哪个?	基础必会
Q038	RDB 与 AOF 同时开启，Redis 重启优先使用什么?	中级
Q039	AOF everysec 是不是绝对只丢 1 秒数据?	高级
Q040	Redis 8.10 的 BACKUP 有什么意义?	新版加分

Q031 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

RDB 是什么?

01 面试标准回答 · 先说结论

RDB 是某一时间点 Redis 数据集的紧凑快照。后台保存通常 fork 子进程写快照，父进程继续服务；适合备份、灾备和较快全量恢复，但两次快照之间的写在故障时可能丢失。

02 原理拆解 · 为什么成立

RPO 取决于快照周期;

RTO 通常较好。

BGSAVE 不是完全无影响：fork 会同步执行，写密集时 COW 会额外复制页面、抬高 RSS。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：持久化题先问 RPO/RTO，再比较 RDB/AOF，通常比背命令更专业。

04 易错点 / Gotchas

不要这样答

不要说 BGSAVE 完全不阻塞；不要只比较文件大小而忽略 fork/COW。

05 工程落地 · 如何把答案变成方案

持久化题先问 RPO/RTO，再比较 RDB/AOF，通常比背命令更专业。

一句话拿分

RDB 是某一时间点 Redis 数据集的紧凑快照。

Q032 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

SAVE 和 BGSAVE 有什么区别?

01 面试标准回答 · 先说结论

`SAVE` 同步生成 RDB，会阻塞正常命令处理；`BGSAVE` fork 子进程写 RDB，父进程继续服务，但 fork、页表复制、COW 仍可能产生延迟和内存峰值。

02 原理拆解 · 为什么成立

大内存实例 fork 时间与页表规模相关；

磁盘越慢，child 存活越久，COW 窗口越长。

监控 `latest_fork_usec`、RDB 状态、RSS 峰值很重要。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手: 区分“写文件在后台”与“fork 动作仍在前台同步发生”。

04 易错点 / Gotchas

不要这样答

不要看到“background”就理解成零影响；不要在内存紧张时频繁触发 BGSAVE。

05 工程落地 · 如何把答案变成方案

区分“写文件在后台”与“fork 动作仍在前台同步发生”。

一句话拿分

`SAVE` 同步生成 RDB，会阻塞正常命令处理；`BGSAVE` fork 子进程写 RDB，父进程继续服务，但 fork、页表复制、COW 仍可能产生延迟和内存峰值。

Q033 | 中级 | 高频真题型 | 原理 · 边界 · 工程

什么是 Copy-On-Write?

01 面试标准回答 · 先说结论

fork 后 parent/child 起初共享物理页；parent 修改某页时，OS 为它复制新页，child 继续看到 fork 时刻的旧页。这样快照无需立刻复制整个数据集，但写越多，额外页越多。

02 原理拆解 · 为什么成立

50GB 数据集 fork 不代表瞬间多 50GB，但持久化期间大量随机写可能让 COW 占用显著上涨。

THP、大页和 allocator 都会影响实际成本。

03 高频追问链 · 面试官会继续问什么

追问 1 fork 为什么不是“瞬间零成本”？

追问 2 BGSAVE 期间写流量大为什么会额外吃内存?

答题抓手：容量规划必须预留 COW headroom，并结合写速率和持久化时长评估峰值。

04 易错点 / Gotchas

不要这样答

不要说 COW“没有额外内存”；不要按 steady-state used_memory 刚好配机器。

05 工程落地 · 如何把答案变成方案

容量规划必须预留 COW headroom，并结合写速率和持久化时长评估峰值。

一句话拿分

fork 后 parent/child 起初共享物理页；parent 修改某页时，OS 为它复制新页，child 继续看到 fork 时刻的旧页。

Q034 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

AOF 是什么?

01 面试标准回答 · 先说结论

AOF 记录能重建数据状态的写操作，重启时回放。与 RDB 周期快照相比，它能把最近写丢失窗口压得更小，但带来持续磁盘写、fsync、文件增长和 rewrite 成本。

```
appendonly yes
appendfsync everysec
```

02 原理拆解 · 为什么成立

常见 `appendfsync everysec` 是 durability 和性能折中；

`always` 更强但写延迟更高。

磁盘慢时 fsync/rewrite 可能形成 latency spike。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：高分点是把 AOF 放在 RPO、磁盘延迟和恢复时间三个维度分析。

04 易错点 / Gotchas

不要这样答

不要把 everysec 解释为数学上“绝对最多丢 1 秒”；不要把 AOF 直接等价成关系数据库 WAL。

05 工程落地 · 如何把答案变成方案

高分点是把 AOF 放在 RPO、磁盘延迟和恢复时间三个维度分析。

一句话拿分

AOF 记录能重建数据状态的写操作，重启时回放。

Q035 | 中级 | 高频真题型 | 原理 · 边界 · 工程

AOF 为什么需要 rewrite?

01 面试标准回答 · 先说结论

AOF 记录历史操作，但恢复只需要最终状态。一个 counter `INCR` 十亿次，不需要永久保留十亿条命令；rewrite 根据当前数据状态生成更紧凑的可恢复表示，降低文件大小和恢复时间。

02 原理拆解 · 为什么成立

rewrite 不是 gzip，而是语义级重写。

rewrite 期间新写还在发生，因此需要 base 和增量日志安全衔接；

大型 rewrite 也会涉及 fork、COW 和磁盘 I/O。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：用“10 亿次 INCR 最终可重建成一个基线状态”举例最直观。

04 易错点 / Gotchas

不要这样答

不要把 rewrite 理解成普通压缩；不要忽略 rewrite 期间新写如何保存。

05 工程落地 · 如何把答案变成方案

用“10 亿次 INCR 最终可重建成一个基线状态”举例最直观。

一句话拿分

AOF 记录历史操作，但恢复只需要最终状态。

Q036 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Redis 7 以后的 Multi-Part AOF 是什么?

01 面试标准回答 · 先说结论

Redis 7 起 AOF 采用 base AOF + incremental AOF + manifest 的 multi-part 结构。rewrite 时生成新的 base，新的写进入增量文件，manifest 决定恢复时哪些文件有效及顺序。

02 原理拆解 · 为什么成立

它降低旧式 rewrite 的重复写/缓冲压力，让 AOF 生命周期更可管理。

Redis 8.10 的 `BACKUP` 也基于 MP-AOF 体系，说明持久化运维已进入多文件管理时代。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：新版高分点：能说出 base/incr/manifest 三件套及它解决的 rewrite 痛点。

04 易错点 / Gotchas

不要这样答

不要继续按 Redis 6 的单文件 AOF 描述新版本；不要手工随意修改 manifest。

05 工程落地 · 如何把答案变成方案

新版高分点：能说出 base/incr/manifest 三件套及它解决的 rewrite 痛点。

一句话拿分

Redis 7 起 AOF 采用 base AOF + incremental AOF + manifest 的 multi-part 结构。

Q037 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

RDB 和 AOF 应该选哪个?

01 面试标准回答 · 先说结论

没有万能答案。纯 cache 可以不持久化；重要状态可考虑 RDB + AOF：RDB 适合快照/备份/较快恢复，AOF 缩小最近写丢失窗口。最终由 RPO、RTO、写负载、磁盘和数据可重建性决定。

02 原理拆解 · 为什么成立

同时开启不等于自动获得强一致：复制/failover 是另一条链路。

持久化越强通常写延迟、磁盘成本越高。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：面试开场先问 RPO/RTO，通常比先背配置更像系统设计。

04 易错点 / Gotchas

不要这样答

不要机械回答“AOF 更安全所以永远选 AOF”；不要忽略数据是否可从 DB/日志重建。

05 工程落地 · 如何把答案变成方案

面试开场先问 RPO/RTO，通常比先背配置更像系统设计。

一句话拿分

没有万能答案。

Q038 | 中级 | 高频真题型 | 原理 · 边界 · 工程

RDB 与 AOF 同时开启，Redis 重启优先使用什么?

01 面试标准回答 · 先说结论

AOF 可用时通常优先按 AOF 恢复，因为其更新粒度比周期 RDB 更细，更接近故障前状态。真正重要的是理解当前版本 MP-AOF manifest、文件完整性和恢复流程，而不是只背一句“优先 AOF”。

02 原理拆解 · 为什么成立

备份必须定期做恢复演练，文件存在并不等于能恢复；

还要考虑异地、加密、保留周期、版本兼容。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手: 高分回答把“有备份”升级成“恢复演练已通过”。

04 易错点 / Gotchas

不要这样答

不要只会背优先级；不要从未验证备份可恢复。

05 工程落地 · 如何把答案变成方案

高分回答把“有备份”升级成“恢复演练已通过”。

一句话拿分

AOF 可用时通常优先按 AOF 恢复，因为其更新粒度比周期 RDB 更细，更接近故障前状态。

Q039 | 高级 | 高频真题型 | 原理 · 边界 · 工程

AOF everysec 是不是绝对只丢 1 秒数据?

01 面试标准回答 · 先说结论

不是。`everysec` 是 fsync 策略，不是形式化强保证；真实故障窗口还受 OS 调度、磁盘、崩溃时点和实现影响。更重要的是，单机 AOF 落盘并不消除异步复制/failover 的写丢失窗口。

02 原理拆解 · 为什么成立

持久化 durability 与复制 consistency 是两条不同链路。

客户端收到 OK 不代表所有 replica 和磁盘都同步；

`WAIT`/更强 fsync 可增强耐久，但仍不能把 Redis 简单宣称成线性一致数据库。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：这题本质是区分“典型经验窗口”和“形式化保证”。

04 易错点 / Gotchas

不要这样答

不要把 everysec 当 SLA；不要只从单机 fsync 推导全系统一致性。

05 工程落地 · 如何把答案变成方案

这题本质是区分“典型经验窗口”和“形式化保证”。

一句话拿分

不是。

Q040 | 新版加分 | 高频真题型 | 原理 · 边界 · 工程

Redis 8.10 的 BACKUP 有什么意义?

01 面试标准回答 · 先说结论

Redis 8.10 新增 `BACKUP`，提供基于 multi-part AOF 的 node-side backup/restore 能力。它反映 Redis 持久化/运维仍在演进，现代面试不能只停留在 `BGSAVE` 和旧单文件 AOF。

02 原理拆解 · 为什么成立

节点侧备份不等于灾难恢复；

仍需把备份复制到独立故障域、验证恢复、控制权限和版本兼容。

8.10.1 还是 2026 年 8 月安全更新，生产应把功能升级与安全维护一起看。

03 高频追问链 · 面试官会继续问什么

追问 1 这个机制在高并发或故障时最容易失效在哪里?

追问 2 如果目标从“性能优先”改成“数据安全优先”，方案需要怎么变?

答题抓手：新版题不必死记参数，重点是能把 BACKUP 放回 RPO/RTO 和 MP-AOF 框架。

04 易错点 / Gotchas

> **不要这样答**
>
> 不要看到 BACKUP 就以为自动完成跨地域容灾；不要忽略备份文件的可恢复性验证。

05 工程落地 · 如何把答案变成方案

新版题不必死记参数，重点是能把 BACKUP 放回 RPO/RTO 和 MP-AOF 框架。

> **一句话拿分**
>
> Redis 8.10 新增 `BACKUP`，提供基于 multi-part AOF 的 node-side backup/restore 能力。

CHAPTER 05

复制与 Sentinel 高可用

掌握异步复制的写安全边界、PSYNC/backlog，以及 Sentinel 的主观/客观下线和选举语义。

> **本章目标**
>
> 完成 Q041–Q050 后，应能在不依赖命令记忆的情况下解释核心机制，并能够说明至少一个“什么时候不该这么做”的边界。

题号	题目	难度
Q041	Redis 主从复制是同步还是异步?	基础必会
Q042	Redis 全量复制是怎样发生的?	中级
Q043	什么是 PSYNC?	中级
Q044	replication backlog 是干什么的?	中级
Q045	WAIT 能不能保证 Redis 强一致?	高级
Q046	Redis Sentinel 是干什么的?	基础必会
Q047	Sentinel 的 SDOWN 和 ODOWN 是什么?	中级
Q048	Sentinel quorum 和 majority 有什么区别?	高级
Q049	Sentinel 为什么推荐奇数个?	中级
Q050	Sentinel 会不会脑裂?	高级

Q041 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Redis 主从复制是同步还是异步?

01 面试标准回答 · 先说结论

Redis 默认 replication 是异步的：primary 执行写并回复客户端，不需要等待所有 replicas 应用写。收益是低延迟和高吞吐，代价是故障时存在已确认写丢失窗口。

02 原理拆解 · 为什么成立

replication lag 不只意味着 replica 读旧数据，也决定 failover 后能保留多少最新写。

`WAIT` 可以让客户端等待一定副本确认，但它不是共识协议。

03 高频追问链 · 面试官会继续问什么

追问 1 异步复制为什么允许已确认写在 failover 后消失?

追问 2 WAIT 能把它变成强一致吗?

答题抓手：这是 Sentinel/Cluster 一致性问题的地基，后续所有 HA 回答都应回到“异步复制”这个事实。

04 易错点 / Gotchas

不要这样答

不要说“有副本就绝不会丢写”；不要把 replica read 当强一致读。

05 工程落地 · 如何把答案变成方案

这是 Sentinel/Cluster 一致性问题的地基，后续所有 HA 回答都应回到“异步复制”这个事实。

一句话拿分

Redis 默认 replication 是异步的：primary 执行写并回复客户端，不需要等待所有 replicas 应用写。

Q042 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Redis 全量复制是怎样发生的?

01 面试标准回答 · 先说结论

当 replica 无法做部分同步时，需要 full sync：primary 准备并传输完整数据集，replica 加载，同时 primary 继续接收的新写要被缓冲，随后再补齐增量。

02 原理拆解 · 为什么成立

大实例 full sync 会同时消耗 fork/COW、CPU、网络、磁盘和 replica 加载内存。

频繁 full sync 往往意味着 backlog 太小、网络不稳或 replica 反复重连。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手：运维中把 full sync 次数/流量作为重要 HA 指标，异常上升应立即追根因。

04 易错点 / Gotchas

不要这样答

不要把 full sync 简化成“复制一个文件”；不要忽略同步期间的新写如何补齐。

05 工程落地 · 如何把答案变成方案

运维中把 full sync 次数/流量作为重要 HA 指标，异常上升应立即追根因。

一句话拿分

当 replica 无法做部分同步时，需要 full sync：primary 准备并传输完整数据集，replica 加载，同时 primary 继续接收的新写要被缓冲，随后再补齐增量。

Q043 | 中级 | 高频真题型 | 原理 · 边界 · 工程

什么是 PSYNC?

01 面试标准回答 · 先说结论

PSYNC 支持 partial resynchronization。replica 短暂断线后携带复制身份与 offset 重连；如果缺失的复制流仍在 primary 的 backlog 中，只补 gap，否则退化为 full sync。

02 原理拆解 · 为什么成立

它把“网络抖动后重传几十 GB”降为“补缺失的几 MB/GB”。

能否 partial sync 取决于缺失字节范围，而不只是断线秒数。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手：可用容量思路：backlog >= 峰值复制字节率 × 希望覆盖的断线时间 × 安全系数。

04 易错点 / Gotchas

不要这样答

不要直接说“断线 10 秒一定能 PSYNC”；不要忽略 replid/offset/backlog。

05 工程落地 · 如何把答案变成方案

可用容量思路：backlog >= 峰值复制字节率 × 希望覆盖的断线时间 × 安全系数。

一句话拿分

PSYNC 支持 partial resynchronization。

Q044 | 中级 | 高频真题型 | 原理 · 边界 · 工程

replication backlog 是干什么的?

01 面试标准回答 · 先说结论

它是 primary 维护的一段近期复制命令流历史，用固定内存换断线恢复效率。replica 重连时若缺失 offset 仍在 backlog，就增量补齐；否则必须全量同步。

02 原理拆解 · 为什么成立

backlog 太小会把轻微网络抖动升级成昂贵 full sync；

太大则占常驻内存。

估算应基于复制 bytes/s 而非 QPS，因为 value 大小差异很大。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手：这是一道很适合做容量计算的题：把写入峰值带宽和可容忍断线窗口明确量化。

04 易错点 / Gotchas

不要这样答

不要凭经验写死 1MB/10MB；不要只看请求数不看字节吞吐。

05 工程落地 · 如何把答案变成方案

这是一道很适合做容量计算的题：把写入峰值带宽和可容忍断线窗口明确量化。

一句话拿分

它是 primary 维护的一段近期复制命令流历史，用固定内存换断线恢复效率。

Q045 | 高级 | 高频真题型 | 原理 · 边界 · 工程

WAIT 能不能保证 Redis 强一致?

01 面试标准回答 · 先说结论

不能。`WAIT` 让客户端在写后等待一定数量 replicas 确认收到复制流，可提高写在故障时被保留的概率，但官方明确说明它不会把 Redis 转成强一致 CP 系统。

```
SET order:42 paid
WAIT 2 100
```

02 原理拆解 · 为什么成立

它不是共识协议;

replica ACK 也不一定等于磁盘 fsync。

timeout 后业务仍要决定失败、重试或降级，网络分区/failover 仍有复杂窗口。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手: 一句话拿分: “WAIT 提高 durability/write safety，但不能改变 Redis 的一致性模型。”

04 易错点 / Gotchas

不要这样答

不要说 WAIT=同步复制=强一致；不要忽略 timeout 后客户端的语义。

05 工程落地 · 如何把答案变成方案

一句话拿分: “WAIT 提高 durability/write safety，但不能改变 Redis 的一致性模型。”

一句话拿分

不能。

Q046 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Redis Sentinel 是干什么的?

01 面试标准回答 · 先说结论

Sentinel 为非 Cluster Redis 提供监控、故障判断、自动 failover 和配置发现。它解决“一个 primary + replicas 如何自动高可用”，不负责数据分片。

02 原理拆解 · 为什么成立

故障后 Sentinel 选择合适 replica 提升为新 primary，并让其他 replicas 重新跟随；

客户端应通过 Sentinel 获取当前 primary，不能把地址永久写死。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手：生产 HA 看 failure domain，不只看进程数量。

04 易错点 / Gotchas

不要这样答

不要把 Sentinel 当分片方案；不要把三个 Sentinel 都放同一宿主机/同一故障域。

05 工程落地 · 如何把答案变成方案

生产 HA 看 failure domain，不只看进程数量。

一句话拿分

Sentinel 为非 Cluster Redis 提供监控、故障判断、自动 failover 和配置发现。

Q047 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Sentinel 的 SDOWN 和 ODOWN 是什么?

01 面试标准回答 · 先说结论

SDOWN 是单个 Sentinel 根据超时形成的“主观下线”；ODOWN 是足够多 Sentinel 对 primary 不可达形成的“客观下线”。它把本地怀疑与集体确认分开。

02 原理拆解 · 为什么成立

一个 Sentinel 网络异常可能产生 SDOWN，但不能单独推进完整 failover。

达到 ODOWN 之后仍要完成 leader/majority 授权等后续步骤。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手：把这题类比“本地事实 vs 集体事实”，能体现分布式系统思维。

04 易错点 / Gotchas

不要这样答

不要把 SDOWN/ODOWN 解释成 replica 的两个状态等级；不要认为 ODOWN 后无需选举。

05 工程落地 · 如何把答案变成方案

把这题类比“本地事实 vs 集体事实”，能体现分布式系统思维。

一句话拿分

SDOWN 是单个 Sentinel 根据超时形成的“主观下线”；ODOWN 是足够多 Sentinel 对 primary 不可达形成的“客观下线”。

Q048 | 高级 | 高频真题型 | 原理 · 边界 · 工程

Sentinel quorum 和 majority 有什么区别?

01 面试标准回答 · 先说结论

quorum 用于判断 primary 是否达到 ODOWN 所需的赞同数；真正执行 failover 的 Sentinel leader 还需要多数 Sentinel 授权。例如 5 个 Sentinel、quorum=2，2 个可触发 ODOWN，但 failover 授权仍需要 majority=3。

02 原理拆解 · 为什么成立

quorum 是配置项，可以小于 majority；

majority 的存在降低少数网络分区单独推进冲突 failover 的风险。

03 高频追问链 · 面试官会继续问什么

追问 1 quorum 与 failover authorization majority 为什么是两件事?

追问 2 5 Sentinel、quorum=2 时各需要多少票?

答题抓手：用 5 节点、quorum=2 的具体算例回答最清晰。

04 易错点 / Gotchas

不要这样答

不要把 quorum 和 majority 当同一个数字；不要只背“奇数节点”而不懂授权机制。

05 工程落地 · 如何把答案变成方案

用 5 节点、quorum=2 的具体算例回答最清晰。

一句话拿分

quorum 用于判断 primary 是否达到 ODOWN 所需的赞同数；真正执行 failover 的 Sentinel leader 还需要多数 Sentinel 授权。

Q049 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Sentinel 为什么推荐奇数个?

01 面试标准回答 · 先说结论

奇数不是 Redis 硬性规则，而是多数派系统里常见的资源效率选择。3 个 Sentinel majority=2，可容忍 1 个故障；4 个 majority=3，通常没带来相同成本比例的额外容错。

02 原理拆解 · 为什么成立

真正重要的是独立 failure domain。

三个进程若都在一台机器/一个 AZ，同一故障会同时带走它们，逻辑多实例并不等于真实高可用。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手： HA 图里同时画进程与 failure domain，面试表现会明显更成熟。

04 易错点 / Gotchas

不要这样答

不要把“奇数”当魔法规则；不要忽略机架/AZ/网络故障域。

05 工程落地 · 如何把答案变成方案

HA 图里同时画进程与 failure domain，面试表现会明显更成熟。

一句话拿分

奇数不是 Redis 硬性规则，而是多数派系统里常见的资源效率选择。

Q050 | 高级 | 高频真题型 | 原理 · 边界 · 工程

Sentinel 会不会脑裂?

01 面试标准回答 · 先说结论

不能说“绝不会”。网络分区后多数派一侧可能选出新 primary，而旧 primary 在少数分区仍可能短时间接受客户端连接；这些写在拓扑收敛后可能被丢弃。

02 原理拆解 · 为什么成立

quorum/majority 降低错误 failover，但不提供共识数据库级线性一致。

客户端服务发现必须跟随新主；

关键外部副作用还可用 fencing/version 等上层机制防止旧主迟到写。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手：高级回答要同时描述“控制平面选主”和“数据平面客户端仍向谁写”。

04 易错点 / Gotchas

不要这样答

不要用“有 majority 就不会双主/丢写”过度承诺；不要只讨论 Redis 内部不看客户端实际连接。

05 工程落地 · 如何把答案变成方案

高级回答要同时描述“控制平面选主”和“数据平面客户端仍向谁写”。

一句话拿分

不能说“绝不会”。

CHAPTER 06

Redis Cluster 与水平扩展

从 16,384 slots、MOVED/ASK、Hash Tag 到 Resharding 和故障转移，能解释 Cluster 的一致性边界。

> **本章目标**
>
> 完成 Q051–Q060 后，应能在不依赖命令记忆的情况下解释核心机制，并能够说明至少一个“什么时候不该这么做”的边界。

题号	题目	难度
Q051	Redis Cluster 为什么是 16384 个 slot?	基础必会
Q052	Redis Cluster 为什么不用一致性哈希?	中级
Q053	Cluster Client 怎么知道 Key 在哪个节点?	基础必会
Q054	MOVED 和 ASK 有什么区别?	中级
Q055	什么是 Hash Tag?	基础必会
Q056	为什么 Cluster 多 Key 操作经常报 CROSSSLOT?	基础必会
Q057	Redis Cluster Resharding 怎么工作?	中级
Q058	Cluster 怎么检测节点故障?	中级
Q059	Cluster 是强一致系统吗?	高级
Q060	Redis Cluster 最少为什么经常部署 6 个实例?	基础必会

Q051 | 基础必会 | 高频真题型 | 原理 • 边界 • 工程

Redis Cluster 为什么是 16384 个 slot?

01 面试标准回答 • 先说结论

Cluster 把 keyspace 固定划分为 16384 hash slots，key 先映射到 slot，再由 slot 映射到 primary。扩缩容主要迁移 slot ownership 和 slot 内 keys，而不是让客户端按节点数直接重算所有 key。

```
slot = CRC16(key) mod 16384
```

02 原理拆解 • 为什么成立

基本映射是 `CRC16(key) mod 16384`，hash tag 是例外。

slot 也是集群状态传播和在线 reshard 的管理单元。

03 高频追问链 • 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手：记住核心心智模型：key -> slot -> node。

04 易错点 / Gotchas

不要这样答

不要说 Redis Cluster 用一致性哈希；不要认为 slot 数运行时可随意改变。

05 工程落地 • 如何把答案变成方案

记住核心心智模型：key -> slot -> node。

一句话拿分

Cluster 把 keyspace 固定划分为 16384 hash slots，key 先映射到 slot，再由 slot 映射到 primary。

Q052 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Redis Cluster 为什么不用一致性哈希?

01 面试标准回答 · 先说结论

固定 hash slots 让 ownership 成为显式、可迁移、可广播的集群状态。客户端可以缓存 16384 slots 到节点的映射，节点也能明确返回 MOVED/ASK。

02 原理拆解 · 为什么成立

一致性哈希适合某些客户端分片，但 Redis Cluster 还要服务端协作 failover、在线 reshard、slot 状态传播；固定 slot 管理粒度更直接。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手: 高分答案说明“不是一致性哈希”之后继续解释 slot 对在线迁移和客户端路由的价值。

04 易错点 / Gotchas

不要这样答

不要把 Cluster 叫“一致性哈希实现”；也不要说扩容完全不搬数据。

05 工程落地 · 如何把答案变成方案

高分答案说明“不是一致性哈希”之后继续解释 slot 对在线迁移和客户端路由的价值。

一句话拿分

固定 hash slots 让 ownership 成为显式、可迁移、可广播的集群状态。

Q053 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Cluster Client 怎么知道 Key 在哪个节点?

01 面试标准回答 · 先说结论

Cluster-aware client 计算 key slot，并缓存 slot -> node map；请求发错节点时 Redis 返回 `MOVED slot host:port`，客户端刷新路由并重试。稳定状态不应每次都先随机问节点。

02 原理拆解 · 为什么成立

客户端启动可获取拓扑；

拓扑变化后要刷新缓存。

普通非 cluster-aware client 往往不能正确处理重定向；

简单四层 LB 也不能替代 slot-aware routing。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手：性能成熟度体现在稳定时期“尽量一次命中正确节点”。

04 易错点 / Gotchas

不要这样答

不要以为 Redis Cluster 有中心 proxy；不要收到 MOVED 后只重试却不更新映射。

05 工程落地 · 如何把答案变成方案

性能成熟度体现在稳定时期“尽量一次命中正确节点”。

一句话拿分

Cluster-aware client 计算 key slot，并缓存 slot -> node map；请求发错节点时 Redis 返回 `MOVED slot host:port`，客户端刷新路由并重试。

Q054 | 中级 | 高频真题型 | 原理 · 边界 · 工程

MOVED 和 ASK 有什么区别?

01 面试标准回答 · 先说结论

`MOVED` 表示 slot 已永久由另一个节点服务，客户端应更新 slot map；`ASK` 主要在 resharding 期间出现，只要求下一次请求临时去目标节点，并先发送 `ASKING`，不能据此永久改 ownership。

```
-MOVED 3999 10.0.0.3:6379
-ASK   3999 10.0.0.4:6379
```

02 原理拆解 · 为什么成立

迁移中同一个 slot 的一部分 key 仍在源节点、一部分已到目标节点，因此需要 ASK 的“一次性临时跳转”；

迁移完成后才会转成稳定 MOVED。

03 高频追问链 · 面试官会继续问什么

追问 1 MOVED 为什么要更新 slot map，而 ASK 通常不更新?

追问 2 ASKING 为什么只影响下一条命令?

答题抓手： 建议能画 A=MIGRATING、B=IMPORTING、客户端 ASKING 的三方时序。

04 易错点 / Gotchas

不要这样答

不要把 ASK 当普通临时网络错误；不要忽略 `ASKING`。

05 工程落地 · 如何把答案变成方案

建议能画 A=MIGRATING、B=IMPORTING、客户端 ASKING 的三方时序。

一句话拿分

`MOVED` 表示 slot 已永久由另一个节点服务，客户端应更新 slot map；`ASK` 主要在 resharding 期间出现，只要求下一次请求临时去目标节点，并先发送 `ASKING`，不能据此永久改 ownership。

Q055 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

什么是 Hash Tag?

01 面试标准回答 · 先说结论

Hash Tag 让 Redis 只对 key 中 `{...}` 片段计算 slot，例如 `order:{1001}:info` 与 `order:{1001}:items` 会落到同一 slot，从而支持某些多 key 命令、事务和脚本。

```
order:{1001}:info
order:{1001}:items
order:{1001}:payment
```

02 原理拆解 · 为什么成立

它的代价是牺牲分布自由度。

若把整个大租户或所有 key 都用同一 tag，会形成 Hot Slot，失去 Cluster 水平分片价值。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手： Hash Tag 的本质是“用局部共址换多 key 原子/便利”，tag 粒度要与业务事务边界匹配。

04 易错点 / Gotchas

不要这样答

不要给所有 key 加 `{global}`；不要只解决 CROSSSLOT 却制造负载倾斜。

05 工程落地 · 如何把答案变成方案

Hash Tag 的本质是“用局部共址换多 key 原子/便利”，tag 粒度要与业务事务边界匹配。

一句话拿分

Hash Tag 让 Redis 只对 key 中 `{...}` 片段计算 slot，例如 `order:{1001}:info` 与 `order:{1001}:items` 会落到同一 slot，从而支持某些多 key 命令、事务和脚本。

Q056 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

为什么 Cluster 多 Key 操作经常报 CROSSSLOT?

01 面试标准回答 · 先说结论

Redis Cluster 不为普通多 key 命令自动做跨节点分布式事务。key 位于不同 slots 时，MGET、集合运算、事务或脚本可能因 CROSSSLOT 失败。

02 原理拆解 · 为什么成立

解决方式是 hash tag 共址、客户端按节点拆分并行请求、或重新设计模型。

客户端拆分后得到的是应用层聚合，不再具备单 Redis 命令的整体原子性。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手：这题背后的设计哲学：Redis Cluster 不隐藏跨 shard 协调的复杂度和延迟。

04 易错点 / Gotchas

不要这样答

不要把 CROSSSLOT 当客户端 bug；不要拆分请求后仍宣称整体原子。

05 工程落地 · 如何把答案变成方案

这题背后的设计哲学：Redis Cluster 不隐藏跨 shard 协调的复杂度和延迟。

一句话拿分

Redis Cluster 不为普通多 key 命令自动做跨节点分布式事务。

Q057 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Redis Cluster Resharding 怎么工作?

01 面试标准回答 · 先说结论

在线 reshard 时源节点把 slot 标记 MIGRATING，目标节点标记 IMPORTING，并逐个迁移该 slot 的 keys。迁移期间部分 key 在源、部分在目标，所以客户端会遇到 ASK；完成后 ownership 更新并进入 MOVED 稳定状态。

02 原理拆解 · 为什么成立

迁移管理单位是 slot，但传输仍是实际 keys；

Big Key 会显著拉长单 key migration。

在线迁移意味着不停机，不等于零性能影响。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手：扩容前先排查 Big Key、Hot Slot，再制定 slot 迁移速率和可观测指标。

04 易错点 / Gotchas

不要这样答

不要以为 ownership 一改数据就瞬移；不要高峰期无速率控制迁大量 slot。

05 工程落地 · 如何把答案变成方案

扩容前先排查 Big Key、Hot Slot，再制定 slot 迁移速率和可观测指标。

一句话拿分

在线 reshard 时源节点把 slot 标记 MIGRATING，目标节点标记 IMPORTING，并逐个迁移该 slot 的 keys。

Q058 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Cluster 怎么检测节点故障?

01 面试标准回答 · 先说结论

Cluster nodes 通过独立 cluster bus 通信并传播 gossip。节点先形成 PFAIL（可能故障）的本地判断，当足够 primary 对故障形成确认后进入 FAIL，随后可能触发 replica promotion。

02 原理拆解 · 为什么成立

cluster bus 是节点内部协议，不是普通客户端 RESP 路径。

故障检测本质仍是超时速度与误判概率之间的平衡。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手：可类比 Sentinel 的“本地怀疑 -> 集体确认”思想，但强调两套机制不同。

04 易错点 / Gotchas

不要这样答

不要把 Cluster 故障检测说成 Sentinel 在工作；不要认为只通过客户端端口探活。

05 工程落地 · 如何把答案变成方案

可类比 Sentinel 的“本地怀疑 -> 集体确认”思想，但强调两套机制不同。

一句话拿分

Cluster nodes 通过独立 cluster bus 通信并传播 gossip。

Q059 | 高级 | 高频真题型 | 原理 · 边界 · 工程

Cluster 是强一致系统吗?

01 面试标准回答 · 先说结论

不是。官方 Cluster specification 明确使用异步复制，并把 write safety 定义为 best-effort。primary 可能已回复客户端但尚未复制，随后故障，新 primary 缺少这条写，从而丢失 acknowledged write；minority partition 窗口更大。

02 原理拆解 · 为什么成立

Cluster 的主要目标是高性能、可扩展和可接受的写安全，而非共识。

`WAIT` 只能增强写保留概率;

业务需要线性一致时应选择更适合的主存储或把 Redis 限定为派生状态。

03 高频追问链 · 面试官会继续问什么

追问 1 Redis Cluster 在什么故障窗口可能丢 acknowledged write?

追问 2 如果业务不能接受这种窗口，应如何改变架构?

答题抓手：高分句：Redis Cluster 允许 acknowledged writes 在特定故障窗口丢失。

04 易错点 / Gotchas

不要这样答

不要说“有多数派所以强一致”；不要把 HA 和 consistency 混成一个概念。

05 工程落地 · 如何把答案变成方案

高分句：Redis Cluster 允许 acknowledged writes 在特定故障窗口丢失。

一句话拿分

不是。

Q060 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Redis Cluster 最少为什么经常部署 6 个实例?

01 面试标准回答 · 先说结论

常见最小 HA 拓扑是 3 个 primary 分担 slots，每个 primary 至少 1 个 replica，所以 6 个实例。三个 primary 便于多数派和分片，replica 负责节点故障时接管。

02 原理拆解 · 为什么成立

这不是协议硬编码的“必须 6 台”；

更大规模可有更多 primaries/replicas。

primary 与自己的 replica 应跨机架/AZ，且 replica promotion 后要有足够容量承受流量。

03 高频追问链 · 面试官会继续问什么

追问 1 发生网络分区或节点重启时，这个结论还成立吗?

追问 2 哪些状态/指标能帮助你判断当前是否已经进入降级或全量同步?

答题抓手：回答最后强调 failure domain 比进程数更重要。

04 易错点 / Gotchas

不要这样答

不要把“6”当常量；不要 6 个进程都放一台宿主机。

05 工程落地 · 如何把答案变成方案

回答最后强调 failure domain 比进程数更重要。

一句话拿分

常见最小 HA 拓扑是 3 个 primary 分担 slots，每个 primary 至少 1 个 replica，所以 6 个实例。

CHAPTER 07

事务、Lua、Functions 与分布式锁

理解 Redis 的原子性到底保证什么；把 WATCH、脚本和锁放进真实的并发/租约问题中。

本章目标

完成 Q061–Q070 后，应能在不依赖命令记忆的情况下解释核心机制，并能够说明至少一个“什么时候不该这么做”的边界。

题号	题目	难度
Q061	Redis 单个命令为什么可以认为是原子的?	基础必会
Q062	MULTI/EXEC 是什么?	基础必会
Q063	Redis Transaction 支持 rollback 吗?	基础必会
Q064	WATCH 是怎么实现乐观锁的?	中级
Q065	Pipeline 和 Transaction 有什么区别?	基础必会
Q066	Lua Script 为什么经常用于 Redis?	中级
Q067	Lua Script 最大风险是什么?	中级
Q068	Redis Functions 和 Lua EVAL 有什么区别?	中级
Q069	怎么正确实现单 Redis 实例分布式锁?	基础必会
Q070	Redlock 是否绝对安全?	高级

Q061 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Redis 单个命令为什么可以认为是原子的?

01 面试标准回答 · 先说结论

对核心命令执行语义，一个命令不会被其他客户端普通命令插入到中间，因此 `INCR/HINCRBY/SET NX` 等能作为并发 primitive。单命令原子不代表多个 round trip 组成的客户端流程原子。

02 原理拆解 · 为什么成立

`GET -> compute -> SET` 存在 race；

复杂 read-modify-write 可用 Lua/Functions 或 WATCH/MULTI。

原子性也与 durability/replication consistency 不同，命令执行成功不等于绝不丢数据。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：并发题先问“能否压成单个 Redis 命令”，往往是最简单可靠的方案。

04 易错点 / Gotchas

不要这样答

不要把原子性扩展到多个命令；不要把原子等同于持久化。

05 工程落地 · 如何把答案变成方案

并发题先问“能否压成单个 Redis 命令”，往往是最简单可靠的方案。

一句话拿分

对核心命令执行语义，一个命令不会被其他客户端普通命令插入到中间，因此 `INCR/HINCRBY/SET NX` 等能作为并发 primitive。

Q062 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

MULTI/EXEC 是什么?

01 面试标准回答 · 先说结论

`MULTI` 之后命令进入队列，`EXEC` 时按顺序连续执行，并保证这批命令执行期间不被其他客户端普通命令穿插。Redis 事务语义不同于关系数据库完整 ACID。

```
MULTI
SET a 1
INCR b
EXEC
```

02 原理拆解 · 为什么成立

运行时某条命令失败不会自动 rollback 其他成功命令；

配合 WATCH 可做乐观并发控制。

`MULTI` 期间无法像交互式 SQL 事务那样根据前一条真实结果动态分支。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：记忆：Redis transaction 的核心是“排队 + 连续执行”，复杂条件分支通常更适合 Lua/Functions。

04 易错点 / Gotchas

不要这样答

不要把 Redis 事务当 MySQL 事务；不要宣称任一错误都会回滚。

05 工程落地 · 如何把答案变成方案

记忆：Redis transaction 的核心是“排队 + 连续执行”，复杂条件分支通常更适合 Lua/Functions。

一句话拿分

`MULTI` 之后命令进入队列，`EXEC` 时按顺序连续执行，并保证这批命令执行期间不被其他客户端普通命令穿插。

Q063 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Redis Transaction 支持 rollback 吗?

01 面试标准回答 · 先说结论

不支持关系数据库意义的自动 rollback。EXEC 阶段某条命令运行时失败，已经成功执行的其他命令不会被撤销；业务需要通过校验、幂等和补偿处理失败。

02 原理拆解 · 为什么成立

Redis 选择较简单事务模型换取实现和性能。

要实现“要么都不修改”，可把校验和写入放入一个短 Lua/Function 中，在真正修改前完整检查前置条件，但脚本本身也要谨慎处理错误路径。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：面试官常用这题验证是否把“连续执行”误解为“ACID rollback”。

04 易错点 / Gotchas

不要这样答

不要说“事务原子所以必然回滚”；不要忽略错误类型和补偿。

05 工程落地 · 如何把答案变成方案

面试官常用这题验证是否把“连续执行”误解为“ACID rollback”。

一句话拿分

不支持关系数据库意义的自动 rollback。

Q064 | 中级 | 高频真题型 | 原理 · 边界 · 工程

WATCH 是怎么实现乐观锁的?

01 面试标准回答 · 先说结论

WATCH 监视一个或多个 key，在 `WATCH -> 读取/计算 -> MULTI/EXEC` 期间若 key 被别人改动，EXEC 会失败/放弃，客户端需要重新读取并重试，类似 CAS。

```
WATCH balance
GET balance
MULTI
SET balance <new>
EXEC
```

02 原理拆解 · 为什么成立

它不锁住 key，别人仍能正常写；

只是提交时验证前提是否仍成立。

冲突率高时大量 abort/retry 会降低效率，复杂逻辑常可比较 Lua/Functions。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：一句话：WATCH 不阻止别人写，只告诉你“你基于的旧状态已经失效”。

04 易错点 / Gotchas

不要这样答

不要把 WATCH 当悲观互斥锁；EXEC 失败后不能继续用旧读取结果重试。

05 工程落地 · 如何把答案变成方案

一句话：WATCH 不阻止别人写，只告诉你“你基于的旧状态已经失效”。

一句话拿分

WATCH 监视一个或多个 key，在 `WATCH -> 读取/计算 -> MULTI/EXEC` 期间若 key 被别人改动，EXEC 会失败/放弃，客户端需要重新读取并重试，类似 CAS。

Q065 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Pipeline 和 Transaction 有什么区别?

01 面试标准回答 · 先说结论

Pipeline 是网络层批处理，目的是减少 RTT/syscall；Transaction 是执行语义，目的是让 EXEC 中的一组命令连续执行、不被其他客户端穿插。Pipeline 不提供事务隔离，也不保证整体成功。

02 原理拆解 · 为什么成立

Pipeline 能把 N 次往返压缩成少量 RTT，但太大的 batch 会占服务器/客户端输出缓冲并抬高单批尾延迟；

两者可以组合，但解决不同问题。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：面试记忆：Pipeline 优化网络；Transaction 约束执行顺序。

04 易错点 / Gotchas

不要这样答

不要把“一次发送”理解成“一次原子执行”；不要无限扩大 pipeline batch。

05 工程落地 · 如何把答案变成方案

面试记忆：Pipeline 优化网络；Transaction 约束执行顺序。

一句话拿分

Pipeline 是网络层批处理，目的是减少 RTT/syscall；Transaction 是执行语义，目的是让 EXEC 中的一组命令连续执行、不被其他客户端穿插。

Q066 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Lua Script 为什么经常用于 Redis?

01 面试标准回答 · 先说结论

Lua 可把多步 read-modify-write 放到 Redis 服务端一次原子执行，减少 RTT 并避免客户端竞态。例如检查库存、扣减、写去重标记可以放在一个短脚本中。

02 原理拆解 · 为什么成立

脚本适合短小、确定、数据局部的逻辑；

Cluster 中涉及的 keys 仍受同 slot 约束。

原子执行意味着其他请求必须等待脚本结束，所以脚本也可能成为阻塞源。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手： Lua 的定位是“短原子业务片段”，不是在 Redis 中实现整个后端。

04 易错点 / Gotchas

不要这样答

不要把大规模扫描/复杂计算放 Lua；不要认为“服务器端执行一定更快”。

05 工程落地 · 如何把答案变成方案

Lua 的定位是“短原子业务片段”，不是在 Redis 中实现整个后端。

一句话拿分

Lua 可把多步 read-modify-write 放到 Redis 服务端一次原子执行，减少 RTT 并避免客户端竞态。

Q067 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Lua Script 最大风险是什么?

01 面试标准回答 · 先说结论

最大风险是长时间阻塞核心执行路径。一个扫描百万 key、遍历巨型集合或复杂循环的 Lua 即使原子，也会让全实例 p99/p999 延迟爆炸。

02 原理拆解 · 为什么成立

设计时必须给出最坏时间复杂度和集合规模上限，并通过 SLOWLOG、LATENCY、脚本耗时 telemetry 监控。

若原子范围太大，优先重构数据模型而不是简单拆成失去语义的 pipeline。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：脚本评审至少要求 worst-case complexity、最大输入规模和超时/发布回滚策略。

04 易错点 / Gotchas

不要这样答

不要只在小测试数据上判断脚本“很快”；不要使用无界 O(N) 操作。

05 工程落地 · 如何把答案变成方案

脚本评审至少要求 worst-case complexity、最大输入规模和超时/发布回滚策略。

一句话拿分

最大风险是长时间阻塞核心执行路径。

Q068 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Redis Functions 和 Lua EVAL 有什么区别?

01 面试标准回答 · 先说结论

EVAL 更像客户端随请求提交或按 SHA 调用脚本；Redis Functions 提供可注册、持久化、集中管理的服务端函数库，更适合稳定复用的原子逻辑。

02 原理拆解 · 为什么成立

Functions 能减少不同应用携带脚本版本不一致，但仍受长执行阻塞、Cluster key locality、部署升级等约束；它不是通用微服务运行时。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：新版候选人应知道 Functions 的存在，同时保持“短、局部、原子”的使用边界。

04 易错点 / Gotchas

不要这样答

不要说 Functions 不会阻塞；不要忽略函数版本发布、回滚和权限管理。

05 工程落地 · 如何把答案变成方案

新版候选人应知道 Functions 的存在，同时保持“短、局部、原子”的使用边界。

一句话拿分

EVAL 更像客户端随请求提交或按 SHA 调用脚本；Redis Functions 提供可注册、持久化、集中管理的服务端函数库，更适合稳定复用的原子逻辑。

Q069 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

怎么正确实现单 Redis 实例分布式锁?

01 面试标准回答 · 先说结论

基本 acquire 是 `SET lock:key unique-token NX PX ttl`。NX 保证不存在才获取，TTL 防永久死锁，unique token 标识 owner。释放必须原子执行“value 等于自己的 token 才 DEL”。

```
SET lock:order:42 <unique-token> NX PX 30000
# release: compare token + DEL atomically
```

02 原理拆解 · 为什么成立

TTL 本质是 lease，业务可能执行超过租期;

主从 failover 也可能因异步复制出现锁状态丢失。

简单 `GET` 后客户端判断再 `DEL` 有 race，应使用 Lua/Function 原子释放。

03 高频追问链 · 面试官会继续问什么

追问 1 为什么 DEL lock:key 不能直接解锁?

追问 2 进程暂停超过 TTL 后，为什么 fencing token 仍重要?

答题抓手：锁题最重要的不是 NX，而是 lease + owner token + 超时/续租 + fencing。

04 易错点 / Gotchas

不要这样答

不要 `SETNX` 后再单独 `EXPIRE`；不要释放时直接 DEL 不校验 owner。

05 工程落地 · 如何把答案变成方案

锁题最重要的不是 NX，而是 lease + owner token + 超时/续租 + fencing。

一句话拿分

基本 acquire 是 `SET lock:key unique-token NX PX ttl`。

Q070 | 高级 | 高频真题型 | 原理 · 边界 · 工程

Redlock 是否绝对安全?

01 面试标准回答 · 先说结论

不能说绝对安全。Redlock 通过多个独立 Redis、majority 获取、TTL/时间假设降低单点问题，但适用性取决于业务故障模型；进程暂停、网络分区、lease 过期后旧 owner 仍可能操作真实资源。

02 原理拆解 · 为什么成立

更强的工程手段是 fencing token：每次成功租约得到单调递增 token，资源端只接受不小于已见最大 token 的请求，从而拒绝过期 owner 的迟到写。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：高级锁题的终点通常是 fencing，而不是“Lua 释放锁”。

04 易错点 / Gotchas

不要这样答

不要把 Redlock 宣称成形式化绝对互斥；不要只分析 Redis key，不分析被保护的外部资源。

05 工程落地 · 如何把答案变成方案

高级锁题的终点通常是 fencing，而不是“Lua 释放锁”。

一句话拿分

不能说绝对安全。

CHAPTER 08

Pub/Sub、Streams 与消息处理

从 fire-and-forget 到 Consumer Group、PEL、XAUTOCLAIM/XNACK，建立至少一次消费与幂等意识。

> **本章目标**
>
> 完成 Q071–Q080 后，应能在不依赖命令记忆的情况下解释核心机制，并能够说明至少一个“什么时候不该这么做”的边界。

题号	题目	难度
Q071	Redis Pub/Sub 的特点是什么?	基础必会
Q072	Pub/Sub 和 Stream 最大区别是什么?	基础必会
Q073	Redis Stream 的 ID 是什么?	基础必会
Q074	Consumer Group 是什么?	基础必会
Q075	什么是 PEL?	中级
Q076	Consumer 崩溃后 Pending Message 怎么处理?	中级
Q077	Redis Streams 能保证 exactly-once 吗?	高级
Q078	Redis 8.8 XNACK 解决了什么?	新版加分
Q079	Redis Stream 怎么防止无限增长?	中级
Q080	Redis 和 Kafka 应该怎么选?	高级

Q071 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Redis Pub/Sub 的特点是什么?

01 面试标准回答 · 先说结论

Pub/Sub 是实时广播模型：publisher 发布到 channel，在线 subscribers 立即接收。它简单、低延迟，但普通 Pub/Sub 不保存历史，离线订阅者不会在重连后补收期间消息。

02 原理拆解 · 为什么成立

适合在线通知、缓存失效、实时事件提示；

不适合必须可靠处理、可重放的订单/支付事件。

ACK、pending、replay 不是普通 Pub/Sub 核心语义。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：如果需求写着“不能丢、要重放”，普通 Pub/Sub 基本应排除。

04 易错点 / Gotchas

不要这样答

不要把 Pub/Sub 当持久消息队列；不要忽略慢订阅者和客户端输出缓冲。

05 工程落地 · 如何把答案变成方案

如果需求写着“不能丢、要重放”，普通 Pub/Sub 基本应排除。

一句话拿分

Pub/Sub 是实时广播模型：publisher 发布到 channel，在线 subscribers 立即接收。

Q072 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Pub/Sub 和 Stream 最大区别是什么?

01 面试标准回答 · 先说结论

Pub/Sub 是 fire-and-forget 在线广播；Streams 是可持久 append-only log，支持历史、Consumer Group、PEL、ACK、claim/replay。选择的核心问题是消费者离线时消息是否仍要存在和恢复。

02 原理拆解 · 为什么成立

同一 Stream 上不同 groups 有独立消费状态；

同一 group 内消息通常分摊给消费者。

Streams 更可靠，但也不能简单等价 Kafka 或 exactly-once。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：记住三个关键词：history + PEL + ACK。

04 易错点 / Gotchas

不要这样答

不要只说“Stream 更可靠”而不讲 PEL/ACK；不要说 Stream 自动 exactly-once。

05 工程落地 · 如何把答案变成方案

记住三个关键词：history + PEL + ACK。

一句话拿分

Pub/Sub 是 fire-and-forget 在线广播；Streams 是可持久 append-only log，支持历史、Consumer Group、PEL、ACK、claim/replay。

Q073 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Redis Stream 的 ID 是什么?

01 面试标准回答 · 先说结论

Stream entry ID 典型形态为 `milliseconds-sequence`，同一 Stream 内单调递增，可作为日志位置和范围查询锚点，支持 `XRANGE/XREVRANGE`。

02 原理拆解 · 为什么成立

毫秒部分与时间相关，sequence 处理同毫秒多消息等情况；

ID 不是全局 UUID，也不能直接推出跨节点全局时间顺序。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：业务可保存 last seen ID 做增量读取，但消费组的 pending ownership 仍由 PEL 管理。

04 易错点 / Gotchas

不要这样答

不要把 Stream ID 当随机主键；不要用它推导跨集群严格全序。

05 工程落地 · 如何把答案变成方案

业务可保存 last seen ID 做增量读取，但消费组的 pending ownership 仍由 PEL 管理。

一句话拿分

Stream entry ID 典型形态为 `milliseconds-sequence`，同一 Stream 内单调递增，可作为日志位置和范围查询锚点，支持 `XRANGE/XREVRANGE`。

Q074 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Consumer Group 是什么?

01 面试标准回答 · 先说结论

Consumer Group 让多个 consumers 共同处理同一 Stream。同一 group 内新消息通常只交给一个 consumer，实现负载分担；不同 groups 则有独立消费位置和 PEL，可各自处理完整消息流。

02 原理拆解 · 为什么成立

这适合 payment-workers、analytics-workers 等独立订阅。

consumer name 应稳定可识别，方便恢复 pending ownership。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：一句话：组内竞争消费，组间独立订阅。

04 易错点 / Gotchas

不要这样答

不要把 group 理解成“组内广播”；不要忽略 pending 积压。

05 工程落地 · 如何把答案变成方案

一句话：组内竞争消费，组间独立订阅。

一句话拿分

Consumer Group 让多个 consumers 共同处理同一 Stream。

Q075 | 中级 | 高频真题型 | 原理 · 边界 · 工程

什么是 PEL?

01 面试标准回答 · 先说结论

PEL（Pending Entries List）记录已经交付给 group 某 consumer、但尚未 XACK 的消息，包括 ownership、投递次数、idle 等状态，是崩溃恢复的核心。

```
XREADGROUP GROUP workers c1 COUNT 10 STREAMS orders >
XACK orders workers 1712345678901-0
```

02 原理拆解 · 为什么成立

`XREADGROUP` 返回并不等于业务成功;

通常业务副作用完成后再 ACK。

PEL 持续增长常意味着未 ACK、消费失败或代码 bug，应使用 XPENDING 监控。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：Streams 的可靠性关键不是“能读消息”，而是“知道哪些消息尚未完成”。

04 易错点 / Gotchas

不要这样答

不要读到消息就立即 ACK 后再做高风险业务；不要没有 pending 监控。

05 工程落地 · 如何把答案变成方案

Streams 的可靠性关键不是“能读消息”，而是“知道哪些消息尚未完成”。

一句话拿分

PEL（Pending Entries List）记录已经交付给 group 某 consumer、但尚未 XACK 的消息，包括 ownership、投递次数、idle 等状态，是崩溃恢复的核心。

Q076 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Consumer 崩溃后 Pending Message 怎么处理?

01 面试标准回答 · 先说结论

崩溃后未 ACK 消息仍在 PEL。其他消费者可以用 `XPENDING` 查看，按 idle 时间用 `XCLAIM` 转移；`XAUTOCLAIM` 可自动扫描并领取超过阈值的 pending。

```
XPENDING orders workers
XAUTOCLAIM orders workers c2 60000 0-0
```

02 原理拆解 · 为什么成立

claim 会让消息再次处理，所以业务必须幂等；

idle 阈值过小会抢走仍在正常执行的长任务，过大则恢复太慢。

还要为 poison message 设计最大重试和 dead-letter。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：可靠消费真正难点是“重复投递是否安全”，不是 claim 命令本身。

04 易错点 / Gotchas

不要这样答

不要无脑高频抢所有 pending；不要让重复执行造成重复扣款等副作用。

05 工程落地 · 如何把答案变成方案

可靠消费真正难点是“重复投递是否安全”，不是 claim 命令本身。

一句话拿分

崩溃后未 ACK 消息仍在 PEL。

Q077 | 高级 | 高频真题型 | 原理 · 边界 · 工程

Redis Streams 能保证 exactly-once 吗?

01 面试标准回答 · 先说结论

不能简单宣称 exactly-once。Consumer Group + ACK 更自然是 at-least-once：业务成功后、ACK 前崩溃会再次投递；若先 ACK 再处理，又会有 at-most-once 丢处理风险。

02 原理拆解 · 为什么成立

工程上用 event_id、数据库唯一约束、幂等状态机，让消息可以重复，但业务效果只生效一次。

所谓 effect-level exactly-once 通常依赖外部存储原子幂等，而不是 Redis 单独提供。

03 高频追问链 · 面试官会继续问什么

追问 1 为什么消费成功但 ACK 前崩溃会重复?

追问 2 如何做到业务效果上的 exactly-once?

答题抓手：把 delivery semantics 和 business effect semantics 分开讲，是高级消息题的关键。

04 易错点 / Gotchas

不要这样答

不要说“有 XACK 就 exactly-once”；不要把重试当异常而非协议设计。

05 工程落地 · 如何把答案变成方案

把 delivery semantics 和 business effect semantics 分开讲，是高级消息题的关键。

一句话拿分

不能简单宣称 exactly-once。

Q078 | 新版加分 | 高频真题型 | 原理 · 边界 · 工程

Redis 8.8 XNACK 解决了什么?

01 面试标准回答 · 先说结论

`XNACK` 允许 consumer 主动释放自己持有的 pending message，使其立即可重新分配，而不是只能等待 idle 超时后再由 XCLAIM/XAUTOCLAIM 恢复。

02 原理拆解 · 为什么成立

它适合优雅下线、资源不足、临时外部依赖失败等“消费者明确知道自己处理不了”的场景；

突然 crash 时仍需 XAUTOCLAIM 等被动恢复。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：新版价值是把“主动失败”与“被动超时 reclaim”两个恢复路径补齐。

04 易错点 / Gotchas

不要这样答

不要把 XNACK 当删除消息；不要认为 nack 后就没有重复处理风险。

05 工程落地 · 如何把答案变成方案

新版价值是把“主动失败”与“被动超时 reclaim”两个恢复路径补齐。

一句话拿分

`XNACK` 允许 consumer 主动释放自己持有的 pending message，使其立即可重新分配，而不是只能等待 idle 超时后再由 XCLAIM/XAUTOCLAIM 恢复。

Q079 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Redis Stream 怎么防止无限增长?

01 面试标准回答 · 先说结论

Stream 是追加日志，必须设计 retention。常用 `XADD ... MAXLEN ~ N` 或 `XTRIM` 控制长度，也可按时间/ID 保留；trim 解决日志体量，但还要考虑 Consumer Group 的 pending 引用和恢复窗口。

02 原理拆解 · 为什么成立

retention 条数不能拍脑袋，应按峰值 msgs/s × 最长可恢复秒数 × 安全系数估算。

近似 trim 通常更高效，长期 TB/PB 历史则更适合专用日志系统。

03 高频追问链 · 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：容量公式能明显提升回答质量：retention ≈ peak rate × replay window。

04 易错点 / Gotchas

不要这样答

不要只设置 MAXLEN 不算消息速率；不要把历史删得比消费者最大离线时间还短。

05 工程落地 · 如何把答案变成方案

容量公式能明显提升回答质量：retention ≈ peak rate × replay window。

一句话拿分

Stream 是追加日志，必须设计 retention。

Q080 | 高级 | 高频真题型 | 原理 • 边界 • 工程

Redis 和 Kafka 应该怎么选?

01 面试标准回答 • 先说结论

不要用“Redis 快、Kafka 慢”选型。Streams 擅长低延迟、中等规模、与 Redis 状态紧密结合的任务流；Kafka 是分布式持久 commit log，擅长大规模分区、长 retention、海量 replay 和数据管道生态。

02 原理拆解 • 为什么成立

TB/PB 历史、按天/月保留、跨团队数据流通常更自然用 Kafka；

已有 Redis 且任务短、延迟敏感、数据量可控时 Streams 很有吸引力。

二者也可组合。

03 高频追问链 • 面试官会继续问什么

追问 1 这个方案提供的是原子性、隔离性、至少一次，还是强一致？不要混淆哪些概念?

追问 2 如果客户端在关键步骤之间崩溃，应如何恢复或补偿?

答题抓手：选型题从 retention、吞吐、replay、消费模型、运维生态五个需求维度出发。

04 易错点 / Gotchas

不要这样答

不要只比较单条消息延迟；不要忽略存储成本、分区扩展和重放生态。

05 工程落地 • 如何把答案变成方案

选型题从 retention、吞吐、replay、消费模型、运维生态五个需求维度出发。

一句话拿分

不要用“Redis 快、Kafka 慢”选型。

CHAPTER 09

性能诊断、运维与安全

把 SLOWLOG、LATENCY、CPU、内存、fork、Swap、ACL/TLS 和可观测性组成排障路径。

> **本章目标**
>
> 完成 Q081–Q090 后，应能在不依赖命令记忆的情况下解释核心机制，并能够说明至少一个“什么时候不该这么做”的边界。

题号	题目	难度
Q081	为什么生产环境不要使用 KEYS *?	基础必会
Q082	SCAN 会不会漏 Key 或重复 Key?	中级
Q083	Redis 延迟突然从 1 ms 变成 500 ms，怎么排查?	高级
Q084	SLOWLOG 记录的是网络耗时吗?	基础必会
Q085	Pipeline 为什么能提高吞吐?	基础必会
Q086	Redis CPU 100%，最先看什么?	高级
Q087	Redis OOM 了怎么办?	高级
Q088	为什么 Redis 不应该使用 Swap?	基础必会
Q089	Redis 应该怎么做安全控制?	基础必会
Q090	Redis 监控最重要的指标有哪些?	基础必会

Q081 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

为什么生产环境不要使用 KEYS *?

01 面试标准回答 · 先说结论

`KEYS *` 需要遍历整个 keyspace，key 数量巨大时会长时间占用核心执行路径，拖高所有客户端延迟。生产遍历应使用 SCAN 增量迭代并控制速率。

```
# avoid
KEYS user:*

# prefer incremental iteration
SCAN 0 MATCH user:* COUNT 1000
```

02 原理拆解 · 为什么成立

O(N) 在小实例可能很快，但线上风险来自 N 不受控；

即使 pattern 很窄也可能扫描大量 key。

SCAN 不是快照，但把大扫描拆成多个小批次。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：线上默认禁用/审计 KEYS，运维扫描用 SCAN + 限速。

04 易错点 / Gotchas

不要这样答

不要在线用 KEYS 查 Big Key；不要以为 pattern 越具体就一定不会全扫描。

05 工程落地 · 如何把答案变成方案

线上默认禁用/审计 KEYS，运维扫描用 SCAN + 限速。

一句话拿分

`KEYS *` 需要遍历整个 keyspace，key 数量巨大时会长时间占用核心执行路径，拖高所有客户端延迟。

Q082 | 中级 | 高频真题型 | 原理 · 边界 · 工程

SCAN 会不会漏 Key 或重复 Key?

01 面试标准回答 · 先说结论

SCAN 是增量迭代而非数据库快照。在遍历期间 keyspace 发生增删会影响观察结果，重复返回是允许的，因此调用方必须幂等或去重；cursor=0 才表示一轮迭代完成。

02 原理拆解 · 为什么成立

`COUNT` 是工作量/返回数量提示，不是严格条数保证。

SCAN 也有 CPU/网络成本，不能因为“非阻塞”就无限高频扫描。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：需要严格在线审计/快照语义时，不要把 SCAN 结果当事务一致视图。

04 易错点 / Gotchas

不要这样答

不要把 cursor 当 offset；不要假设每个 key 恰好出现一次。

05 工程落地 · 如何把答案变成方案

需要严格在线审计/快照语义时，不要把 SCAN 结果当事务一致视图。

一句话拿分

SCAN 是增量迭代而非数据库快照。

Q083 | 高级 | 高频真题型 | 原理 · 边界 · 工程

Redis 延迟突然从 1 ms 变成 500 ms，怎么排查?

01 面试标准回答 · 先说结论

按 client -> network -> Redis -> OS 分层，而不是一上来只看 SLOWLOG。客户端查连接池、GC、DNS；网络查 RTT/丢包；Redis 查 SLOWLOG、LATENCY、commandstats、Big/Hot Key；OS/持久化查 fork、AOF fsync、swap、CPU steal、磁盘抖动。

```
SLOWLOG GET 128
LATENCY LATEST
LATENCY DOCTOR
INFO commandstats
```

02 原理拆解 · 为什么成立

SLOWLOG 干净不代表端到端没问题;

延迟尖刺若与 `latest_fork_usec`、AOF event、expire/eviction cycle 同时出现，应重点查内核和后台任务。

务必关注 p99/p999 而非平均值。

03 高频追问链 · 面试官会继续问什么

追问 1 如果 SLOWLOG 没有异常，但客户端 p99 很高，你下一步查什么?

追问 2 fork/AOF/expire/eviction 如何进入同一套延迟诊断?

答题抓手: 排障回答最好呈现一条因果路径和验证步骤，而不是堆工具名。

04 易错点 / Gotchas

不要这样答

不要看到 500ms 就直接扩容；不要只看 Redis CPU。

05 工程落地 · 如何把答案变成方案

排障回答最好呈现一条因果路径和验证步骤，而不是堆工具名。

一句话拿分

按 client -> network -> Redis -> OS 分层，而不是一上来只看 SLOWLOG。

Q084 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

SLOWLOG 记录的是网络耗时吗?

01 面试标准回答 · 先说结论

不是。SLOWLOG 主要记录 Redis 处理命令本身的执行时间，不等于客户端完整 RTT。连接池等待、网络、DNS、客户端 GC、大 reply 传输都可能让用户很慢但 SLOWLOG 很干净。

02 原理拆解 · 为什么成立

因此需要把应用 tracing/APM 与 Redis 内部指标结合。

大 value 返回时服务器计算可能不慢，但网络传输/客户端反序列化很重。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：排障时先问“这个指标覆盖请求路径的哪一段”。

04 易错点 / Gotchas

不要这样答

不要把 SLOWLOG 当 end-to-end profiler；不要凭它为空就断言 Redis 链路没问题。

05 工程落地 · 如何把答案变成方案

排障时先问“这个指标覆盖请求路径的哪一段”。

一句话拿分

不是。

Q085 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Pipeline 为什么能提高吞吐?

01 面试标准回答 · 先说结论

Pipeline 把多条命令连续发送后批量收回复，把 N 次 request/response RTT 降为少量 RTT，同时减少 syscall 和用户态/内核态切换，尤其适合很多独立的小命令。

```
without pipeline:  N requests ~= N x RTT
with pipeline:     N commands ~= ~1 x RTT + server work
```

02 原理拆解 · 为什么成立

吞吐提升常来自网络而非 Redis 算法改变。

batch 太大时客户端和服务器要暂存大量 reply，会增加内存与单批尾延迟；

Cluster pipeline 还应按节点/slot 分组。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：优化前先测 RTT 占比，再 benchmark batch size，不要照抄固定 1000/10000。

04 易错点 / Gotchas

不要这样答

不要无限增大 batch；不要把 Pipeline 当事务。

05 工程落地 · 如何把答案变成方案

优化前先测 RTT 占比，再 benchmark batch size，不要照抄固定 1000/10000。

一句话拿分

Pipeline 把多条命令连续发送后批量收回复，把 N 次 request/response RTT 降为少量 RTT，同时减少 syscall 和用户态/内核态切换，尤其适合很多独立的小命令。

Q086 | 高级 | 高频真题型 | 原理 · 边界 · 工程

Redis CPU 100%，最先看什么?

01 面试标准回答 · 先说结论

先确认 CPU 消耗在核心命令线程、I/O/search worker 还是 OS 其他任务，再看 `INFO commandstats`、SLOWLOG、LATENCY、Hot Key、大集合操作、Lua/Functions、expire/eviction/defrag。

02 原理拆解 · 为什么成立

单 shard CPU 满时给机器加更多核未必有效，尤其是单 Hot Key 或长串行命令。

要找“高频小成本”和“低频高成本”两类命令，并按 shard 而不是集群平均值定位。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：CPU 排障的核心问题是“谁在花 CPU、为什么现在变贵了”。

04 易错点 / Gotchas

不要这样答

不要直接扩容节点数却不处理 Hot Key；不要只看总 CPU。

05 工程落地 · 如何把答案变成方案

CPU 排障的核心问题是“谁在花 CPU、为什么现在变贵了”。

一句话拿分

先确认 CPU 消耗在核心命令线程、I/O/search worker 还是 OS 其他任务，再看 `INFO commandstats`、SLOWLOG、LATENCY、Hot Key、大集合操作、Lua/Functions、expire/eviction/defrag。

Q087 | 高级 | 高频真题型 | 原理 · 边界 · 工程

Redis OOM 了怎么办?

01 面试标准回答 · 先说结论

先区分达到 `maxmemory` 后 Redis 按策略淘汰/拒写，还是 Linux 因实际 RSS 峰值 OOM kill。后者可能在 `used_memory < maxmemory` 时发生，因为 RSS 还包含碎片、COW、client output buffer、replication buffer 等。

02 原理拆解 · 为什么成立

检查 INFO memory、evictions/rejected writes、RSS、fork/COW、client buffers、Big Key。

maxmemory 不应等于机器 RAM，必须为 OS 和持久化峰值留空间。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：第一句话先问“Redis policy OOM 还是 OS OOM killer”，能快速体现排障框架。

04 易错点 / Gotchas

不要这样答

不要只把 maxmemory 调大；不要看到 OOM 就关持久化而不查根因。

05 工程落地 · 如何把答案变成方案

第一句话先问“Redis policy OOM 还是 OS OOM killer”，能快速体现排障框架。

一句话拿分

先区分达到 `maxmemory` 后 Redis 按策略淘汰/拒写，还是 Linux 因实际 RSS 峰值 OOM kill。

Q088 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

为什么 Redis 不应该使用 Swap?

01 面试标准回答 · 先说结论

Redis 的低延迟来自内存访问。热页被 swap 到磁盘后，下次访问会发生 major page fault，延迟从微秒级内存访问跳到毫秒甚至更高，并形成不可预测的尾延迟抖动。

02 原理拆解 · 为什么成立

swap 可能让进程暂时不死，但会破坏 SLO。

正确方法是合理 maxmemory、容量余量、cgroup/OS 监控和避免内存压力。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：延迟敏感内存数据库的原则是显式容量管理，而不是依赖 swap 承担工作集。

04 易错点 / Gotchas

不要这样答

不要把 swap 当廉价扩容；不要只监控 used_memory。

05 工程落地 · 如何把答案变成方案

延迟敏感内存数据库的原则是显式容量管理，而不是依赖 swap 承担工作集。

一句话拿分

Redis 的低延迟来自内存访问。

Q089 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Redis 应该怎么做安全控制?

01 面试标准回答 · 先说结论

生产最低线包括私网/防火墙、ACL 最小权限、TLS、凭据轮换、危险命令/管理面限制、审计、备份保护和及时安全升级；Redis 不应直接暴露公网。

02 原理拆解 · 为什么成立

ACL 可限制用户命令类别和 key pattern。

Redis 8.10.1 在 2026 年 8 月包含多项安全修复，说明“在内网”也不能成为长期不升级的理由。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：用 defense-in-depth 结构答：网络、身份、权限、传输、补丁、审计、备份。

04 易错点 / Gotchas

不要这样答

不要认为“有密码就能暴露公网”；不要给默认用户全命令权限。

05 工程落地 · 如何把答案变成方案

用 defense-in-depth 结构答：网络、身份、权限、传输、补丁、审计、备份。

一句话拿分

生产最低线包括私网/防火墙、ACL 最小权限、TLS、凭据轮换、危险命令/管理面限制、审计、备份保护和及时安全升级；Redis 不应直接暴露公网。

Q090 | 基础必会 | 高频真题型 | 原理 · 边界 · 工程

Redis 监控最重要的指标有哪些?

01 面试标准回答 · 先说结论

监控要围绕 SLO 与故障模式：p50/p95/p99/p999 latency、ops/s、连接、网络、used_memory/RSS/fragmentation、evictions、hit/miss、CPU、fork、RDB/AOF、replication lag/full sync、cluster state、Big/Hot Key。

02 原理拆解 · 为什么成立

平均延迟会掩盖尾延迟；

hit ratio 99% 也不一定安全，如果总 QPS 很大，1% miss 仍可能压垮 DB。

所有集群指标最好按 shard 展开，避免热点被平均值隐藏。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：顶层看用户 p99/error rate，再下钻资源与 Redis 内部事件。

04 易错点 / Gotchas

不要这样答

不要只看 CPU/内存；不要把命中率百分比脱离绝对 miss QPS 看。

05 工程落地 · 如何把答案变成方案

顶层看用户 p99/error rate，再下钻资源与 Redis 内部事件。

一句话拿分

监控要围绕 SLO 与故障模式：p50/p95/p99/p999 latency、ops/s、连接、网络、used_memory/RSS/fragmentation、evictions、hit/miss、CPU、fork、RDB/AOF、replication lag/full sync、cluster state、Big/Hot Key。

CHAPTER 10

Redis 8.x 与系统设计压轴题

覆盖 Redis 8 新数据类型与命令，并以秒杀、排行榜、限流和生产架构完成综合设计。

> **本章目标**
>
> 完成 Q091–Q100 后，应能在不依赖命令记忆的情况下解释核心机制，并能够说明至少一个“什么时候不该这么做”的边界。

题号	题目	难度
Q091	Redis 8 最大的架构变化是什么？	新版加分
Q092	Vector Set 和 Redis Query Engine Vector Search 怎么选？	新版加分
Q093	Bloom Filter 为什么适合解决缓存穿透？	中级
Q094	Count-Min Sketch、Top-K、t-digest 有什么作用？	新版加分
Q095	Redis 8.8 的 Array 与传统 List 有什么不同？	新版加分
Q096	Redis 8.8 INCREX 为什么值得关注？	新版加分
Q097	设计一个秒杀库存系统，你会怎么用 Redis？	系统设计
Q098	设计一个高并发排行榜系统。	系统设计
Q099	设计一个 Redis Rate Limiter，有哪些算法？	系统设计
Q100	终极题：如果让你设计一个生产级 Redis 架构，你会怎么做？	系统设计

Q091 | 新版加分 | 高频真题型 | 原理 · 边界 · 工程

Redis 8 最大的架构变化是什么?

01 面试标准回答 · 先说结论

Redis 8 的重要变化是把 Search、JSON、Time Series、Bloom/Cuckoo、CMS、Top-K、t-digest 等过去 Redis Stack 能力整合进 Redis Open Source 主发行版，并持续推进 I/O 与 Query Engine 的并行/性能能力。

02 原理拆解 · 为什么成立

现代 Redis 面试边界已从“缓存 + 五大类型”扩展到搜索、向量、时序、概率结构，但持久化、复制、Cluster、Big/Hot Key 等经典原理仍是生产主干。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：把新版能力视为经典 Redis 的增量扩展，而不是替代旧基础。

04 易错点 / Gotchas

不要这样答

不要把 Redis 8 说成“所有命令都完全多线程”；不要只记新命令而忽略一致性和故障边界。

05 工程落地 · 如何把答案变成方案

把新版能力视为经典 Redis 的增量扩展，而不是替代旧基础。

一句话拿分

Redis 8 的重要变化是把 Search、JSON、Time Series、Bloom/Cuckoo、CMS、Top-K、t-digest 等过去 Redis Stack 能力整合进 Redis Open Source 主发行版，并持续推进 I/O 与 Query Engine 的并行/性能能力。

Q092 | 新版加分 | 高频真题型 | 原理 · 边界 · 工程

Vector Set 和 Redis Query Engine Vector Search 怎么选?

01 面试标准回答 · 先说结论

Vector Set 更接近 Redis-native 的向量相似度数据结构，适合以向量近邻为核心的轻量场景；Query Engine 更适合 vector + full text + numeric/tag filters + secondary index 的混合检索。

02 原理拆解 · 为什么成立

RAG 常需要 tenant、时间、标签等过滤与 KNN 联合，这时 Query Engine 更自然；

只做简单相似搜索时 Vector Set API 更直接。

选择还要 benchmark 维度、索引、召回、内存和更新率。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：从 query pattern 选择索引，而不是从“哪个更新”选择产品。

04 易错点 / Gotchas

不要这样答

不要认为“内存数据库就一定是最佳向量库”；不要只比较 ANN 延迟不看 filter 和运维。

05 工程落地 · 如何把答案变成方案

从 query pattern 选择索引，而不是从“哪个更新”选择产品。

一句话拿分

Vector Set 更接近 Redis-native 的向量相似度数据结构，适合以向量近邻为核心的轻量场景；Query Engine 更适合 vector + full text + numeric/tag filters + secondary index 的混合检索。

Q093 | 中级 | 高频真题型 | 原理 · 边界 · 工程

Bloom Filter 为什么适合解决缓存穿透?

01 面试标准回答 · 先说结论

Bloom Filter 用小空间判断“肯定不存在”或“可能存在”。把有效 ID/key 集合编码进 Bloom 后，大量随机不存在请求可在 DB 前被挡住，从而缓解缓存穿透。

02 原理拆解 · 为什么成立

标准 Bloom 有 false positive：可能让不存在的 key 继续查 DB；

通常没有传统意义 false negative。

动态删除/扩容则可能需要 Cuckoo/可计数方案。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：穿透治理最好是参数校验 + Bloom + 负缓存 + 限流的组合，而不是只加一个 Bloom。

04 易错点 / Gotchas

不要这样答

不要说 Bloom 能返回真实 value；不要忽略容量和误判率随填充上升。

05 工程落地 · 如何把答案变成方案

穿透治理最好是参数校验 + Bloom + 负缓存 + 限流的组合，而不是只加一个 Bloom。

一句话拿分

Bloom Filter 用小空间判断“肯定不存在”或“可能存在”。

Q094 | 新版加分 | 高频真题型 | 原理 · 边界 · 工程

Count-Min Sketch、Top-K、t-digest 有什么作用?

01 面试标准回答 · 先说结论

它们都是用近似换内存效率：CMS 估计元素频率，Top-K 维护高频候选，t-digest 估计 p50/p95/p99 等分位数，适合高吞吐流式统计。

02 原理拆解 · 为什么成立

概率结构应明确误差边界。

热门内容/Hot Key 检测适合 Top-K/CMS；

延迟分布适合 t-digest；

需要审计精确值、财务结算时则不应只依赖近似结果。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：先问“能否接受近似”，再决定是否使用概率结构。

04 易错点 / Gotchas

不要这样答

不要把近似结果当精确账本；不要不向业务解释误差。

05 工程落地 · 如何把答案变成方案

先问“能否接受近似”，再决定是否使用概率结构。

一句话拿分

它们都是用近似换内存效率：CMS 估计元素频率，Top-K 维护高频候选，t-digest 估计 p50/p95/p99 等分位数，适合高吞吐流式统计。

Q095 | 新版加分 | 高频真题型 | 原理 · 边界 · 工程

Redis 8.8 的 Array 与传统 List 有什么不同?

01 面试标准回答 · 先说结论

Redis 8.8 新增 Array，面向有序、可索引的字符串集合；传统 List 更自然地表达 deque/双端 push-pop 和阻塞队列。Array 更强调 numeric index 访问。

02 原理拆解 · 为什么成立

两者不是简单替代关系。

过去某些只能把整个数组序列化进 String 的场景，可用 Array 获得更原生的局部操作，但仍要评估命令复杂度、内存、Cluster 和客户端支持。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：新版数据类型依旧遵循“主访问模式驱动”的数据建模原则。

04 易错点 / Gotchas

不要这样答

不要把 Array 当 List 新名字；不要因为新就无脑迁移。

05 工程落地 · 如何把答案变成方案

新版数据类型依旧遵循“主访问模式驱动”的数据建模原则。

一句话拿分

Redis 8.8 新增 Array，面向有序、可索引的字符串集合；传统 List 更自然地表达 deque/双端 push-pop 和阻塞队列。

Q096 | 新版加分 | 高频真题型 | 原理 · 边界 · 工程

Redis 8.8 INCREX 为什么值得关注?

01 面试标准回答 · 先说结论

`INCREX` 把窗口计数器常见的增量、边界和过期控制合并成一个原子命令，减少过去 `INCR + EXPIRE + Lua/transaction` 的样板与竞态，特别适合 fixed/sliding window counter 限流。

```
INCREX ...   # atomic counter + bounds + expiration (Redis 8.8)
```

02 原理拆解 · 为什么成立

它可以在 key 新建时设置 TTL，避免每次请求把窗口续长，并支持 bounds/实际增量返回。

它不是所有限流算法的万能替代，token bucket、精确 sliding log 仍有不同状态模型。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：新版高分点：Redis 正在把高频 Lua pattern 下沉为 native atomic primitive。

04 易错点 / Gotchas

不要这样答

不要把 INCREX 当完整流量治理系统；不要让所有用户共用一个 limiter key 造成 Hot Key。

05 工程落地 · 如何把答案变成方案

新版高分点：Redis 正在把高频 Lua pattern 下沉为 native atomic primitive。

一句话拿分

`INCREX` 把窗口计数器常见的增量、边界和过期控制合并成一个原子命令，减少过去 `INCR + EXPIRE + Lua/transaction` 的样板与竞态，特别适合 fixed/sliding window counter 限流。

Q097 | 系统设计 | 高频真题型 | 原理 · 边界 · 工程

设计一个秒杀库存系统，你会怎么用 Redis?

01 面试标准回答 · 先说结论

秒杀不能只回答 `DECR stock`。完整链路应包括入口限流、用户去重、Redis 原子预扣库存、成功事件进入 Streams/MQ、数据库最终建单、支付超时/失败补偿和幂等。

02 原理拆解 · 为什么成立

Lua/Function 可原子执行“是否重复购买 + stock>0 + 扣减 + 写 reservation/event”。

DB 用唯一约束/状态机保证事件幂等；

Redis 扣库存成功但 DB 失败必须有补偿，不是简单无条件加回库存。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：高分框架：流量控制 -> 原子预扣 -> durable event -> DB 幂等 -> 补偿 -> 监控/对账。

04 易错点 / Gotchas

不要这样答

不要把 Redis 原子扣减等价为全链路分布式事务；不要没有幂等 ID 和补偿状态机。

05 工程落地 · 如何把答案变成方案

高分框架：流量控制 -> 原子预扣 -> durable event -> DB 幂等 -> 补偿 -> 监控/对账。

一句话拿分

秒杀不能只回答 `DECR stock`。

Q098 | 系统设计 | 高频真题型 | 原理 · 边界 · 工程

设计一个高并发排行榜系统。

01 面试标准回答 · 先说结论

中小规模可直接 ZSet；超大规模不能用一个全球 ZSet，应按赛区/地区/时间窗口/hash shard 维护局部榜，再从每个 shard 取局部 Top-K 在应用/聚合层 merge，历史榜按赛季或日期分段。

02 原理拆解 · 为什么成立

实时个人排名与全球 Top-N 可能需要不同读路径。

分片后不必取全量成员，局部候选就足以合并 Top-K；

同分规则也要稳定定义。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手：系统设计的真正考点是“从单 ZSet 扩展到分布式 Top-K”。

04 易错点 / Gotchas

不要这样答

不要只会 `ZREVRANGE 0 99`；不要为了多 key 方便又把所有分片 hash tag 到同一 slot。

05 工程落地 · 如何把答案变成方案

系统设计的真正考点是“从单 ZSet 扩展到分布式 Top-K”。

一句话拿分

中小规模可直接 ZSet；超大规模不能用一个全球 ZSet，应按赛区/地区/时间窗口/hash shard 维护局部榜，再从每个 shard 取局部 Top-K 在应用/聚合层 merge，历史榜按赛季或日期分段。

Q099 | 系统设计 | 高频真题型 | 原理 · 边界 · 工程

设计一个 Redis Rate Limiter，有哪些算法?

01 面试标准回答 · 先说结论

常见 Fixed Window、Sliding Log、Sliding Window Counter、Token Bucket、Leaky Bucket。Fixed Window 状态少但窗口边界可瞬时超发；Sliding Log 精确但内存高；Sliding Counter 折中；Token Bucket 允许可控 burst。

```
Fixed window      -> counter + TTL / INCREX
Sliding log        -> ZSET timestamps
Token bucket       -> tokens + refill time
```

02 原理拆解 · 为什么成立

Redis 8.8 INCREX 强化窗口计数器实现；

ZSet 可实现 sliding log，但必须及时清理旧时间戳。

全局单 limiter key 在高 QPS 下会成为 Hot Key，可按用户/租户/节点分层限流。

03 高频追问链 · 面试官会继续问什么

追问 1 你会用哪些指标或实验验证这个方案，而不是凭经验判断?

追问 2 当 p99、内存或可用性目标冲突时，你会如何做权衡?

答题抓手: 答题按准确度、内存、burst tolerance、跨节点成本四维权衡。

04 易错点 / Gotchas

不要这样答

不要只会 `INCR+EXPIRE`；不要忽略 TTL 生命周期和分布式额度一致性。

05 工程落地 · 如何把答案变成方案

答题按准确度、内存、burst tolerance、跨节点成本四维权衡。

一句话拿分

常见 Fixed Window、Sliding Log、Sliding Window Counter、Token Bucket、Leaky Bucket。

Q100 | 系统设计 | 高频真题型 | 原理 · 边界 · 工程

终极题：如果让你设计一个生产级 Redis 架构，你会怎么做？

01 面试标准回答 · 先说结论

先从 SLO 和故障模型出发：数据是否可重建、QPS/带宽、容量、p99、RPO/RTO、一致性要求，再决定 Cluster 分片、replica/failure domain、持久化、缓存一致性、热点治理、Big Key 限制、客户端策略、监控和演练。

02 原理拆解 · 为什么成立

容量不是 10TB/单机内存这么算，而是 dataset + metadata + fragmentation + buffers + COW headroom + safety margin，并按最热 shard 峰值规划。

HA 要理解异步复制丢写窗口；

一致性用 Cache Aside + retry/CDC/version；

关键副作用用幂等/fencing；

定期演练 primary kill、network partition、disk full、reshard、cache avalanche。

03 高频追问链 · 面试官会继续问什么

追问 1 如何把 1M QPS 和 10TB logical cache 映射成 shard 数量?

追问 2 如何证明你的 HA 不是“只存在于配置文件里”？

答题抓手：终极回答模板：需求/SLO -> 数据模型 -> 分片 -> HA/一致性 -> 持久化 -> 性能 -> 容量 -> 可观测性 -> 故障演练 -> 安全。

04 易错点 / Gotchas

不要这样答

不要拿“3 主 3 从”当起点却没问需求；不要只看集群平均资源；不要把容量配到 95%-100% 没有 failover/reshard 余量。

05 工程落地 · 如何把答案变成方案

终极回答模板：需求/SLO -> 数据模型 -> 分片 -> HA/一致性 -> 持久化 -> 性能 -> 容量 -> 可观测性 -> 故障演练 -> 安全。

一句话拿分

先从 SLO 和故障模型出发：数据是否可重建、QPS/带宽、容量、p99、RPO/RTO、一致性要求，再决定 Cluster 分片、replica/failure domain、持久化、缓存一致性、热点治理、Big Key 限制、客户端策略、监控和演练。

附录 A • 30 个高频追问速查

01. Redis 快到底快在哪里？单线程解释够吗？

02. Big Key 与 Hot Key 如何分别发现和治理？

03. listpack 为什么只适合小对象？

04. ZSet 为什么同时需要 member lookup 与有序范围能力？

05. TTL 到点为什么不等于内存瞬间下降？

06. allkeys-lru 与 volatile-lru 如何选？

07. 缓存击穿为什么本质是请求合并问题？

08. 为什么 Cache Aside 不能声称强一致？

09. BGSAVE 的 fork 和 COW 分别带来什么风险？

10. AOF everysec 的“1 秒”为什么不能承诺为绝对上限？

11. 为什么 Multi-Part AOF 能减少 rewrite 压力？

12. PSYNC 什么时候退化成 full resync？

13. replication backlog 应该怎么定容量？

14. WAIT 为什么不是共识协议？

15. SDOWN、ODOWN、quorum、majority 如何串起来？

16. 为什么 Cluster 是 slots 而不是一致性哈希？

17. MOVED/ASK 在 resharding 中分别表示什么？

18. Hash Tag 如何同时解决 CROSSSLOT 又可能制造 Hot Slot？

19. Redis Cluster 为什么仍可能丢已确认写？

20. MULTI/EXEC 和 SQL transaction 最大的语义差异？

21. WATCH 的乐观锁在 key 过期时会怎样？

22. Pipeline 与 Lua 都能少 RTT，二者的边界是什么？

23. 为什么 Redis 锁需要 unique token？

24. 为什么 lease-based lock 最后还要考虑 fencing？

25. Pub/Sub 与 Streams 如何按可靠性选？

26. PEL 为什么意味着 Streams 更自然地提供 at-least-once？

27. XAUTOCLAIM 与 XNACK 分别解决什么故障？

28. SLOWLOG 没异常但 p99 高，下一步怎么查？

29. used_memory 没到 maxmemory，机器为什么仍可能 OOM？

30. 设计 Redis 架构时如何把 SLO、RPO、RTO 映射到具体配置？

附录 B • 生产故障排查清单

CHECK 延迟突然上升

- 客户端连接池/GC/事件循环与网络 RTT
- SLOWLOG 与 commandstats
- LATENCY LATEST / HISTORY / DOCTOR
- fork、AOF fsync/rewrite、active expire、eviction、defrag
- Big Key、Hot Key、大集合命令、Lua/Functions
- CPU steal、Swap、THP、磁盘与网络异常

CHECK 内存异常

- used_memory 与 RSS 是否明显背离
- mem_fragmentation_ratio 与 allocator 视角
- Big Key / 无界集合 / 无界 Stream
- client output buffer 与 replication buffer
- BGSAVE/AOF rewrite 的 COW headroom
- maxmemory policy 是否符合业务语义

CHECK 复制异常

- connected replicas / replication lag
- master_repl_offset 与 replica offset
- backlog 是否足够覆盖断线窗口
- full resync 是否反复发生
- 网络带宽、fork、RDB 传输/加载压力
- failover 后是否有 acknowledged write 丢失风险

CHECK Cluster 异常

- cluster_state、slot coverage、fail/pfail
- MOVED/ASK 激增是否来自 resharding 或拓扑过期
- slot skew / Hot Slot
- CROSSSLOT 是否来自数据建模
- reshard/migration 是否碰到 Big Key
- 客户端是否正确刷新 slot map

CHECK 缓存异常

- hit ratio 是否突降
- 数据库 QPS 是否同步飙升
- 是否出现单 Hot Key 击穿
- TTL 是否集中导致雪崩
- negative cache / Bloom 是否有效
- 更新 DB 与缓存失效链路是否存在重试/CDC

排障原则：先区分“客户端 / 网络 / 服务端 / 操作系统”，再区分“持续高负载 / 周期性尖刺 / 单 Key 热点 / 后台任务”。没有证据时不要直接重启 Redis；重启可能清掉可观测证据，并诱发全量同步或缓存雪崩。

附录 C • 7 / 14 / 30 天复习计划

7 天冲刺 每天约 90–120 分钟

- D1：Q1–20，数据结构 + 内存；口述每题 60 秒。
- D2：Q21–40，缓存 + 持久化；手画 Cache Aside 与 RDB/AOF。
- D3：Q41–60，复制 + Sentinel + Cluster；重点 MOVED/ASK。
- D4：Q61–80，事务/锁 + Streams；重点故障边界。
- D5：Q81–100，排障 + Redis 8.x + 系统设计。
- D6：只做 30 个追问速查；错题回到原题。
- D7：模拟面试：随机抽 20 题 + Q97–100 全部口述。

14 天稳固 每天约 60–90 分钟

- 前 10 天：每天 10 题，先答后看。
- D11：缓存一致性、锁、Streams 三个专题复盘。
- D12：RDB/AOF/Replication/Cluster 故障链路复盘。
- D13：用 SLOWLOG/LATENCY/INFO 写一份排障 runbook。
- D14：完成 1 次 60 分钟模拟面试并整理错题。

30 天体系化 每天约 45–60 分钟

- Week 1：基础数据结构、encoding、内存。
- Week 2：缓存、持久化、高可用、Cluster。
- Week 3：并发、消息、性能、安全与 Redis 8.x。
- Week 4：系统设计 + 3 次模拟面试 + 错题二刷。
- 最后一天：只复习“一句话拿分”和 Gotchas，不再扩新知识。

附录 D • Redis 版本差异速览

版本	关键变化	面试影响
Redis 5 / 6	Streams 逐步成熟；Redis 6 引入 ACL、I/O threading 等。	传统高频题仍然是底座，但不能把“Redis 单线程”绝对化。
Redis 7.x	Functions、Multi-Part AOF 等成为重要生产知识点。	持久化与服务器端可编程的面试答案需要升级。
Redis 8.0	Search、JSON、Time Series、概率结构等整合进 Redis Open Source；Vector Set 进入新数据类型体系。	“Redis 只有五大类型”的回答已经过时。
Redis 8.8	Array、INCREX、XNACK 等新能力。	数组、窗口计数、Stream pending 主动释放成为新版加分题。
Redis 8.10	Compact Hash、HIMPORT、BACKUP、LMOVEM/BLMOVEM、集合基数类命令等。	关注内存密度、批量导入与运维备份的新路径。

版本意识

面试回答应先说明“经典语义”再补“当前版本变化”。不要把新命令当作必须替代旧方案，也不要用 Redis 5/6 的知识断言 Redis 8.x 的能力边界。

参考资料 • 官方文档优先

01. Redis Open Source 8.10 release notes
https://redis.io/docs/latest/operate/oss_and_stack/stack-with-enterprise/release-notes/redisce/redisos-8.10-release-notes/

02. What’s new in Redis 8.8
https://redis.io/docs/latest/develop/whats-new/8-8/

03. Redis data types
https://redis.io/docs/latest/develop/data-types/

04. OBJECT ENCODING
https://redis.io/docs/latest/commands/object-encoding/

05. Key eviction
https://redis.io/docs/latest/develop/reference/eviction/

06. Persistence
https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/

07. Replication
https://redis.io/docs/latest/operate/oss_and_stack/management/replication/

08. Sentinel
https://redis.io/docs/latest/operate/oss_and_stack/management/sentinel/

09. Redis Cluster specification
https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/

10. Transactions
https://redis.io/docs/latest/develop/using-commands/transactions/

11. Pipelining
https://redis.io/docs/latest/develop/using-commands/pipelining/

12. Distributed locks with Redis
https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/

13. Pub/Sub
https://redis.io/docs/latest/develop/pubsub/

14. Streams
https://redis.io/docs/latest/develop/data-types/streams/

15. Latency optimization
https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/latency/

16. Client-side caching
https://redis.io/docs/latest/develop/clients/client-side-caching/

> **阅读建议**
>
> 遇到版本、命令语义、复制/Cluster 写安全等问题时，优先回到 Redis 官方文档和 Cluster specification。博客和面经适合发现题型，不应替代协议与官方语义。